

Rapport de Stage - Master 2 CRYPTIS

Capgemini

Déploiement automatisé d'une solution PAM et son écosystème dans le cloud

Vincent Joulain

Encadrants Entreprise :	Bastien RICHARD
	Pierre-Antoine JOURDON
Encadrant Faculté :	Philippe GABORIT
Date :	5 septembre 2024

Résumé

Ce stage au sein Capgemini Toulouse, objet de ce rapport, permet l'obtention du diplôme Master CRYPTIS de université de Limoges. L'objectif en a été un déploiement automatisé complet de la solution CyberArk et de son écosystème afin de proposer un support de formation aux consultants souhaitant se former sur cette solution. De plus, cet environnement permettrait aux professionnels du domaine l'essai de nouvelles fonctionnalités et l'apport de correctifs sur des versions déployées. Ce travail réalisé en collaboration avec Tom Berland (étudiant en Master Mathématique) a été l'occasion d'étudier en profondeur la solution CyberArk et la gestion d'accès à privilèges.

Remerciements

Ce projet n'a pu voir le jour qu'à l'initiative de Bastien Richard, Practice Manager et tuteur de stage au sein de la section Identity Access Management chez Capgemini Toulouse et Pierre-Antoine Jourdon, consultant Privileged Access Management au sein de la même entité. Tous deux ont apporté soutien, suivi et expertise tout au long du projet en se rendant disponibles et à l'écoute des problématiques rencontrées, sans cela ce travail n'aurait pas pu être réalisé. Les remerciements sont aussi dirigés vers toute l'équipe PAM de Capgemini Toulouse avec qui il a été possible de convenir de réunions ponctuelles afin de résoudre des situations bloquantes. Enfin, merci à Sebastien Baule, attentif et réactif aux différentes demandes concernant l'environnement de travail proposé par l'IAM Academy.

Table des matières

1 Capgemini l'entreprise **4**
- 1.1 De 1967 à aujourd'hui . . . 4
- 1.2 Et les métiers de la cyber-sécurité ? . . . 4
- 1.3 IAM Academy et besoin d'automatisation . . . 4

2 Contexte du projet **6**
- 2.1 Pourquoi un environnement AzureLAB, quelles différences ? . . . 6
- 2.2 Qu'est-ce que la gestion des accès à privilèges . . . 6
  - 2.2.1 Produits du marché . . . 7
- 2.3 Les solutions d'Infrastructure as Code . . . 8
  - 2.3.1 Etude comparative des outils disponibles . . . 9

3 Sujet et implémentation **10**
- 3.1 Gestion de projet . . . 10
  - 3.1.1 Répartition des tâches . . . 10
- 3.2 Exigences du projet . . . 10
- 3.3 CyberArk . . . 11
  - 3.3.1 Fonctionnement et établissement d'une connexion privilégiée . . . 11
  - 3.3.2 Environnement de déploiement . . . 12
- 3.4 Ansible . . . 13
  - 3.4.1 Prérequis . . . 13
  - 3.4.2 Cahiers de jeu, rôles et tâches . . . 13
  - 3.4.3 Arborescence du projet . . . 15
- 3.5 Environnement de déploiement . . . 16
  - 3.5.1 Echange de clés SSH . . . 16
  - 3.5.2 Infrastructure à clés publiques . . . 16
  - 3.5.3 Annuaire . . . 17
  - 3.5.4 Serveur NTP . . . 18
- 3.6 Composants CyberArk . . . 18
  - 3.6.1 Password Vault . . . 18
  - 3.6.2 Password Vault Web Access . . . 23
  - 3.6.3 Central Policy Manager . . . 24
  - 3.6.4 Privileged Session Manager . . . 24
  - 3.6.5 HTML5 Gateway . . . 25
  - 3.6.6 Privileged Threat Analytics . . . 26
- 3.7 Embarquement des comptes et tests d'utilisation . . . 27
  - 3.7.1 Etape d'onboarding . . . 27
  - 3.7.2 Utilisation de la solution . . . 27

4 Résultats **28**
- 4.1 Fonctionnalités et composants déployés . . . 28
- 4.2 Les concessions . . . 28
- 4.3 Suite pour le projet . . . 28

5 Sujet annexe : Etude de la génération d'IV sur AES-CTR **29**
- 5.1 AES - Counter mode . . . 29
- 5.2 Méthode de création du nonce . . . 30
  - 5.2.1 Compteur et double-compteur . . . 31
  - 5.2.2 Fonctions de hachage . . . 32
  - 5.2.3 Générateur Congruentiel Linéaire . . . 33
- 5.3 Solution adoptée . . . 33

6 Conclusion **34**

7 Glossaire **35**

8 Annexes **37**
- 8.1 Sources . . . 37
- 8.2 Documentation . . . 37

1 Capgemini l'entreprise

Capgemini est une entreprise des services du numérique Française (ESN) basée à Paris regroupant de nombreux établissements en France et dans le monde. Celui de Toulouse, réparti sur 3 sites différents propose, dans la sécurité informatique, des services tels que la gestion d'accès et d'identités, de l'architecture réseaux et cloud Zero trust, des tests d'intrusion ainsi que de l'analyse de risque et conformité au normes en vigueur.

1.1 De 1967 à aujourd'hui

Après avoir reconnu un besoin pour du support technique sur le marché français, Serge Kampf fonde le 1er octobre 1967 la société **Sogeti** "**SO**ciété pour la **G**estion de l'**E**ntreprise et le **T**raitement de l'**I**nformation".

Aujourd'hui, l'entreprise **Capgemini**, issue de l'intégration de **Sogeti** à **CAP** en 1973 ainsi qu'a **Gemini Services** en 1974, est un des leaders mondiaux dans les services du numérique et de l'ingénierie de proximité. La marque **Sogeti** reste toutefois au coeur de l'entreprise, c'est pourquoi elle a été relancée en 2002 avec sa mission d'origine : fournir des services IT avec une forte implication locale. Celle-ci a été réintégrée de nouveau à **Capgemini** en 2018 en tant que **Sogeti, part of Capgemini**.

1.2 Et les métiers de la cyber-sécurité ?

Depuis l'avènement du travail à distance, les employés d'une entreprise ou plus généralement les utilisateurs d'un système informatique ont besoin d'accéder à des ressources privées depuis un réseau externe. Ce besoin nécessite d'établir un moyen de déterminer l'identité des utilisateurs et ainsi pouvoir accorder les droits d'accès correspondants, c'est le rôle de la gestion d'identités et d'accès (IAM), un des composants principaux dans le domaine de la sécurité informatique et aussi un service proposé par **Capgemini Toulouse**.

L'introduction d'une hiérarchie entre les comptes – certains possédant des droits supérieurs aux autres – implique une répartition des risques non uniforme. La gestion de ces comptes à hauts privilèges fait partie d'un sous-composant de l'IAM appelé gestion d'accès à privilèges (PAM).

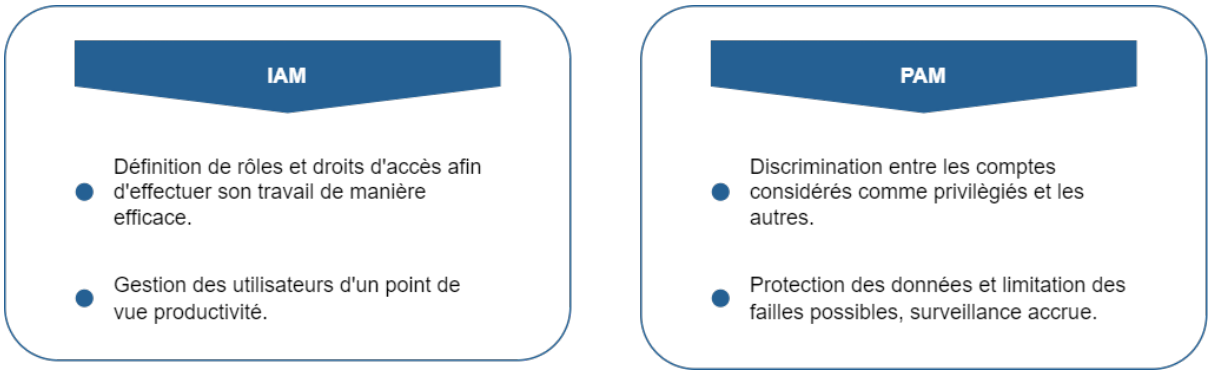


FIGURE 1 – Différences entre IAM et PAM

1.3 IAM Academy et besoin d'automatisation

l'IAM Academy est un dispositif de formation aux métiers de l'IAM proposé en interne par **Capgemini France**. Des environnements AzureLAB sont mis à disposition des professionnels souhaitant passer une certification sur ces métiers ou effectuer des tests sur un environnement isolé. Intégré au service IAM et plus précisément aux solutions PAM, un sujet en lien direct avec l'IAM Academy a été proposé. Tom Berland et moi-même avons fait le choix de travailler ensemble sur le sujet exposé ci-dessous, représentant deux défis principaux : le premier par **l'utilisation de technologies peu familières** et le second par **la complexité de la chaîne technique** afin d'automatiser la solution dans sa globalité.

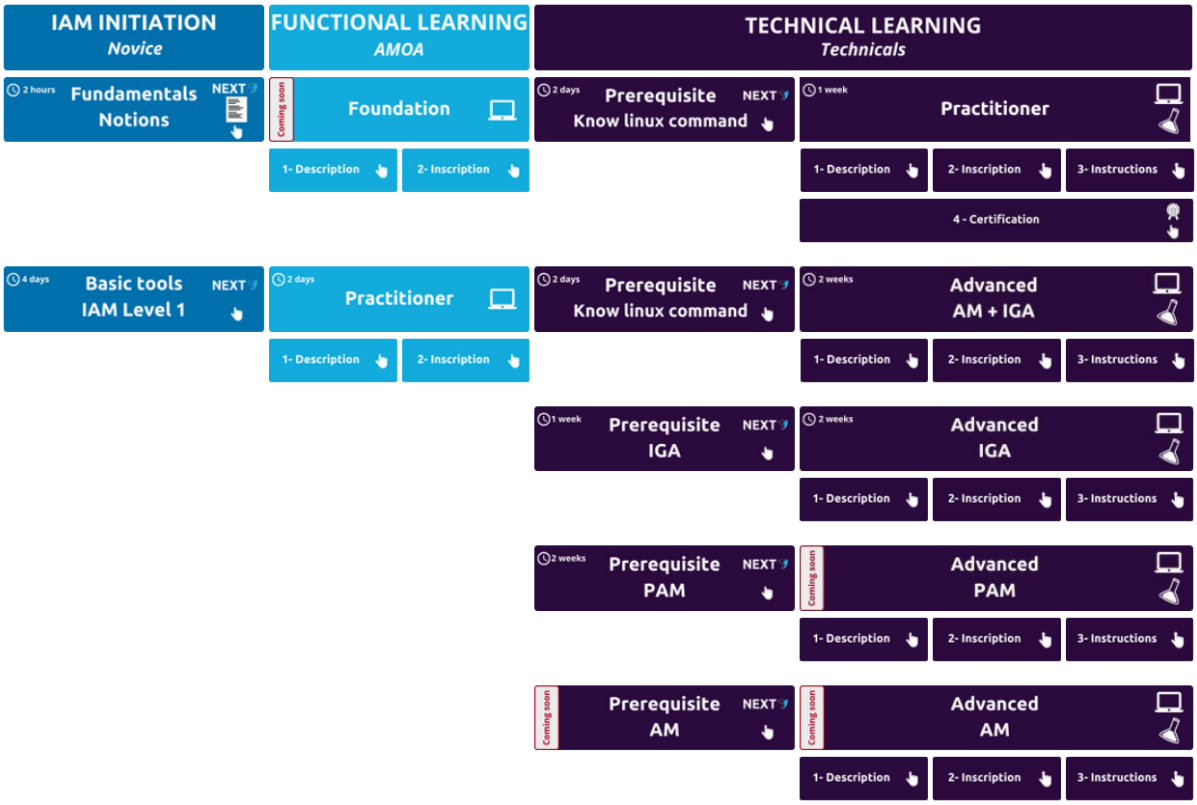


FIGURE 2 – Parcours de formation dispensé par l'IAM Academy

Déploiement CyberArk en contexte AzureLAB par une solution d'IaC

Les consultants de la communauté PAM ont régulièrement besoin de tester des fonctionnalités ou de reproduire des bogues Cyberark en environnement isolé sur des versions différentes du produit (hors environnement client). Le produit Cyberark PAS est constitué d'un cœur (le Vault) ainsi que de composants (PVWA, PSM/P, CPM, PTA, etc.) sur lesquels il est parfois nécessaire d'effectuer une installation sur des machines virtuelles dédiées ; ces machines ne pouvant pas être construites à l'avance et constituer un ensemble instanciable dans AzureLAB. En effet, l'architecture doit pouvoir être installée au besoin et dans un ordre défini, la communication inter-machines reposant sur des adresses IP et non sur des FQDN (limitation du produit.)

Afin que les consultants puissent instancier un ensemble de machines virtuelles Cyberark fonctionnelles dans AzureLAB pour une durée de vie limitée, il a été envisagé de déployer des machines vierges dédiées à Cyberark, une machine AD, une machine MFA (produit à déterminer) et une machine IaC (Ansible ou autre). L'automatisation prendrait en compte le déploiement des produits dans la version choisie (similaire à un déploiement « on premise »), la configuration CyberArk (via l'API), le déploiement de comptes sur les machines virtuelles et enrôlement dans Cyberark ainsi qu'une intégration AD et MFA.

2 Contexte du projet

2.1 Pourquoi un environnement AzureLAB, quelles différences ?

AzureLAB est un service de Microsoft permettant de définir des ensembles de machines réparties sur un ou plusieurs VLAN, appelées Laboratoires (LABs). Chaque LAB peut ensuite être publié afin de permettre à ceux qui le souhaitent d'accéder à l'environnement défini au préalable. Pensé pour la formation, les machines virtuelles sont préinstanciées à partir d'un modèle défini et faciles à réimager. L'IAM Academy a fait le choix d'utiliser et de mettre à disposition des images AzureLAB aux professionnels de la communauté afin de dispenser des formations certifiantes. En revanche, AzureLAB n'est pas une réplique d'un environnement on premise. Certaines différences, bien que non fondamentales, exigent une adaptation de la solution si celle-ci venait à être exportée.

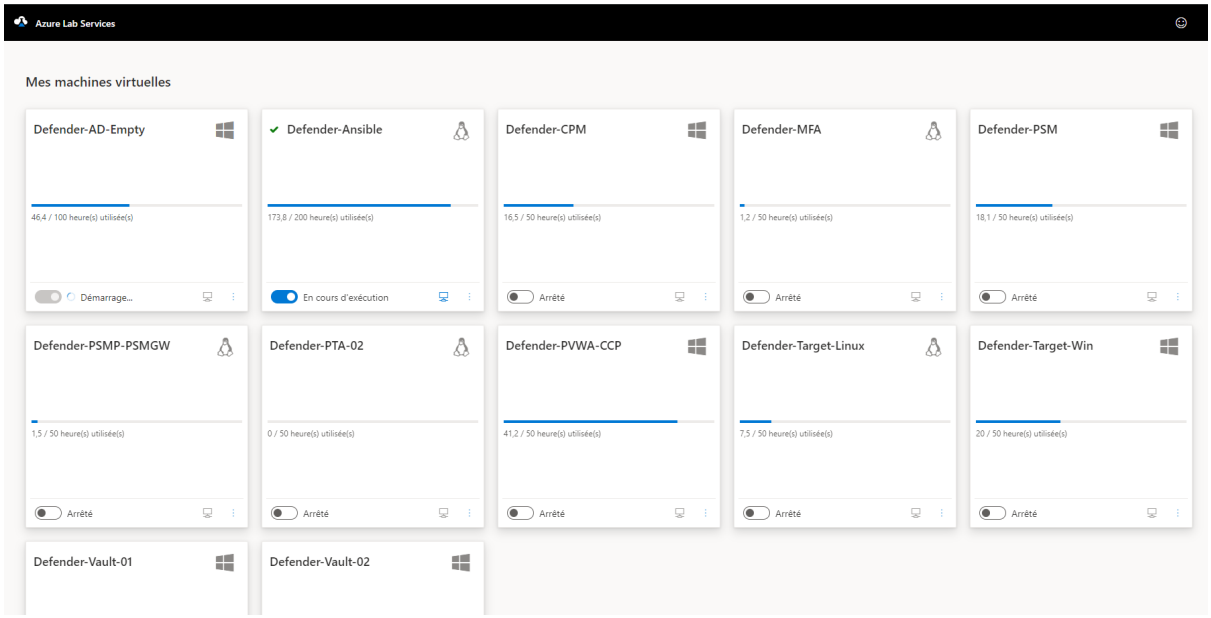


FIGURE 3 – Lab distribué à un étudiant pour la formation Defender proposée par l'IAM Academy

2.2 Qu'est-ce que la gestion des accès à privilèges

La gestion d'accès à privilèges, communément appelée Privileged Access Management ou PAM, est une des briques essentielles de l'IAM. La majorité des attaques cyber concluantes sont issues de la compromission d'un compte à privilèges : administrateur serveur, compte public d'une entreprise ou bien gestionnaire des ressources humaines sont chacun d'une manière ou d'une autre des ressources de grande valeur pour un adversaire cherchant à nuire à l'entreprise.

Afin de protéger l'accès à ces ressources, la mise en place d'une solution PAM peut être envisagée, celle-ci repose sur quatre composants centraux :

- le coffre fort communément appelé Vault dans lequel sont sauvegardés les secrets des comptes administrateurs serveurs,
- le gestionnaire de mots de passe chargé de faire tourner les secrets dans le Vault et les serveurs à atteindre en fonction des politiques de sécurité choisies,
- le gestionnaire d'accès permettant l'authentification de l'utilisateur au travers d'un MFA par exemple,
- le gestionnaire de session qui provisionne l'accès aux serveurs cibles tout en gardant une trace écrite ou vidéo des actions effectuées.

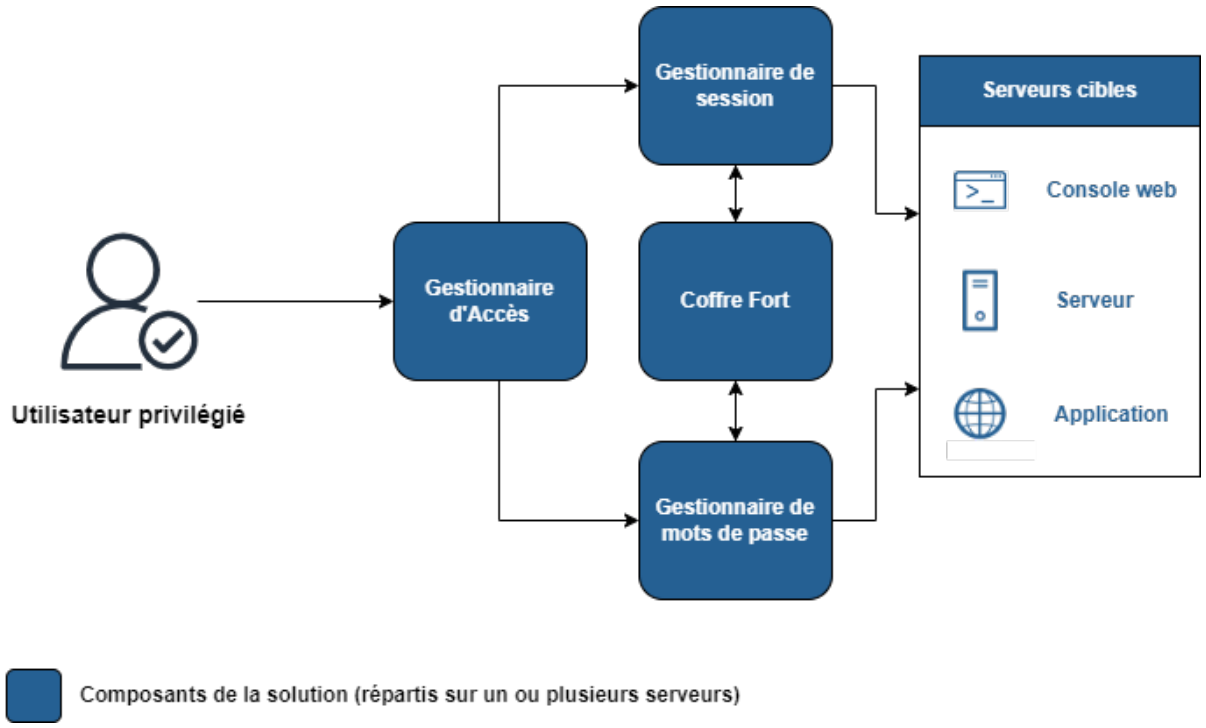


FIGURE 4 – Les quatres composants centraux d'une solution PAM

Ainsi, la compromission d'un compte administrateur sur les différents serveurs requiert ou bien la récupération d'un mot de passe fort (ou clé d'authentification) enregistré sur le Vault et changé régulièrement, ou bien l'usurpation d'un compte utilisateur protégé par MFA ou certificats dont les accès sont restreints et surveillés. En revanche, la centralisation des secrets sur le Vault augmente de manière exponentielle le risque d'attaques sur ce-dernier, le coffre-fort nécessite par conséquent une politique de sécurité drastique.

2.2.1 Produits du marché

Les solutions PAM ont pris une grande ampleur ces dix dernières années. **Cyberark**, apparu en 1999, est leader du marché depuis les années 2010 grâce à la complexité et l'avance des services proposés par l'entreprise comparée aux concurrents : répartition sur des serveurs indépendants de chaque composant, aucune attaque efficace recensée à ce jour sur leur solution, ensemble de fonctionnalités encore indisponibles chez les compétiteurs. Toutefois, il est à noter que **Delinea** ou **BeyondTrust** s'approprient une part de marché de plus en plus conséquente depuis les 5 dernières années.

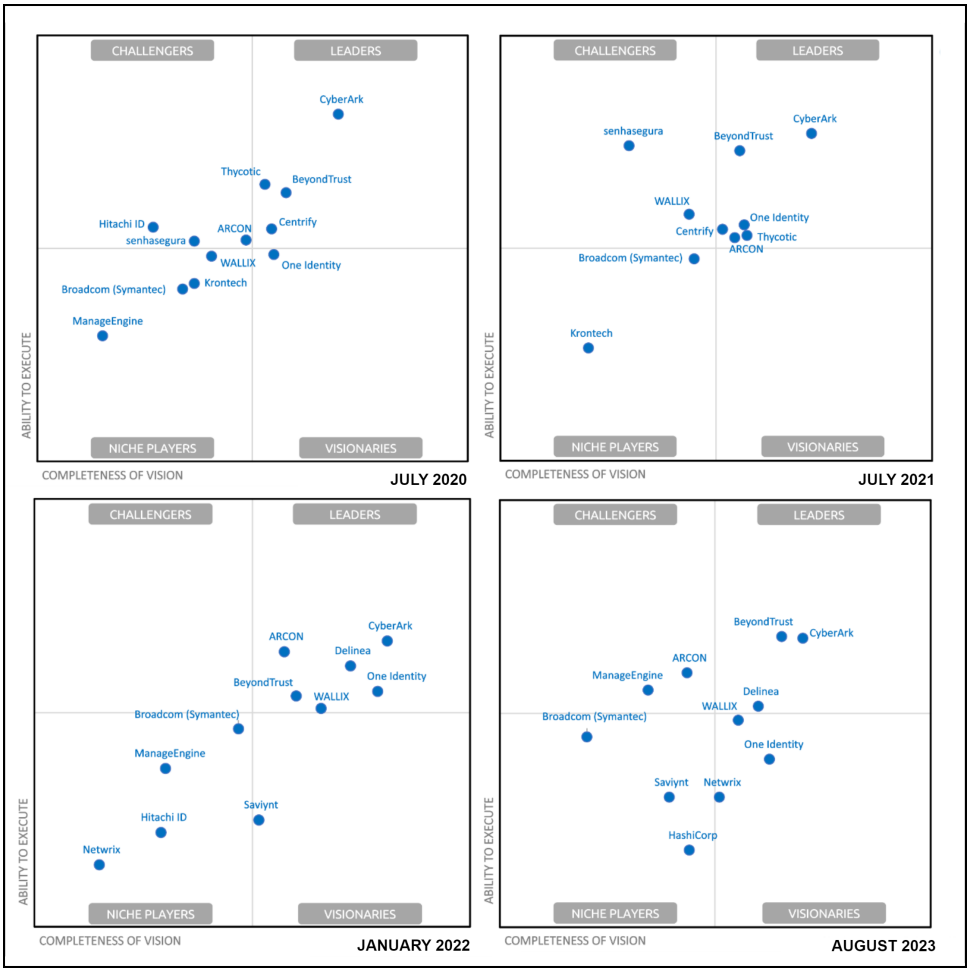


FIGURE 5 – Evolution des solution PAM entre juillet 2020 et août 2023

2.3 Les solutions d'Infrastructure as Code

Dans l'objectif d'automatiser un déploiement / configuration on premise ou on cloud : Ansible, Terraform et Puppet sont parmi d'autres des solutions efficaces. Ces outils étant open-source, populaires et possédant un tiers gratuit peu limité, une comparaison a été nécessaire afin de sélectionner le plus adapté.

Lors de la mise en place d'un service cloud, plusieurs étapes sont essentielles :

— **Provisionnement** des ressources : création de machines virtuelles, attribution des réseaux et gestion du stockage par dessus une infrastructure physique donnée.

— **Installation** logicielle : système d'exploitation, correctifs, mises à jour, applications et configuration.

— **Edition** de l'infrastructure (si besoin) : ajout, modification ou suppression de machines virtuelles et/ou réseaux virtuels, déplacement des ressources computationnelles et du stockage.

— **Supression** de l'infrastructure.

Définir et comprendre ces étapes est un socle indispensable afin de comparer les outils en fonction des avantages et inconvénients qu'ils offrent sur chacune des phases de déploiement, mais aussi permettre d'appréhender au mieux les besoins du projet. Dans la section suivante, une étude comparative de Ansible, Puppet et Terraform est présentée.

2.3.1 Etude comparative des outils disponibles

Terraform est par essence une solution permettant l'automatisation des étapes de provisionnement et de gestion de l'infrastructure cloud. Grâce à son mode de fonctionnement déclaratif et sa capacité à mémoriser les états précédents de l'infrastructure il est un outil adapté afin de supprimer, modifier ou ajouter des machines virtuelles sur un cloud.

Contrairement à Terraform, Ansible fonctionne de manière itérative et ne gère pas le cycle de vie de l'infrastructure cloud. Aucun différentiel ne va être effectué lors du lancement du Ansible Playbook entre l'état actuel et l'état précédent, conduisant dans certains cas à la création de nouveaux playbooks afin d'effectuer les modifications nécessaires. En revanche, Ansible excelle dans l'étape d'installation et de configuration des machines virtuelles, l'outil permet l'exécution d'actions précises et spécifiques (modification et copie de fichiers, installation de paquets, lancement de commandes, etc) pour lesquelles Terraform n'est pas adapté.

Enfin, Puppet est un outil plus similaire à Ansible que Terraform, adapté à l'étape de configuration et d'installation au niveau applicatif et facilement intégrable dans le cloud Azure (au moyen d'extensions), il a été un candidat sérieux à la réalisation de notre projet. Toutefois, en plus d'être un outil plus difficile à prendre en main qu'Ansible par sa configuration plus poussée et son langage plus complexe, il possède dans notre contexte un point négatif non négligeable : **l'utilisation d'agents**. En effet, cela implique la préinstallation de ces agents sur les machines visées allant à l'encontre de l'objectif initial qui était le lancement d'un programme d'automatisation à partir de machines vierges (mis à part la station de travail IaC), de plus, l'installation d'un agent tournant en tâche de fond sur le Vault va à l'encontre des recommandations de sécurité de CyberArk et par conséquent de la plupart des installations réelles, mettant à mal la simulation visée au départ.

Comparaison des outils d'automatisation Ansible, Terraform et Puppet :

	Terraform	**Ansible**	**Puppet**
Objectifs	Provisionnement et gestion de l'infrastructure	Installation et configuration d'applications	Installation et configuration d'applications
Architecture	Client seul	Client seul	Serveur / Client
Sauvegarde d'états	Oui	Non	Non
Paradigme	Déclaratif	Itératif	Déclaratif
Langage utilisé	HCL	YAML	Puppet DSL
Editeur	HashiCorp	RedHat	Perforce

Après étude, Ansible semble tout indiqué pour notre projet et cela pour trois raisons :

— dans l'environnement AzureLAB, les machines virtuelles sont prédéfinies et prêtes à l'emploi une fois le laboratoire publié puis détruites lors de sa suppression, les phases de provisionnement et de gestion des machines ne sont pas nécessaires ;

— le programme IaC est voué à être lancé une unique fois lors du lancement du LAB, effectuer des modifications à l'architecture déployée n'est pas une demande du sujet, une approche itérative sans gestion d'état est parfaitement adaptée ;

— de nombreuses ressources sont déjà disponibles visant à automatiser CyberArk via Ansible, proposées par l'éditeur ou par la communauté (voir PAS Orchestrator).

3 Sujet et implémentation

3.1 Gestion de projet

Le sujet de stage défini et un superviseur assigné (Pierre-Antoine Jourdon), la première étape a été de mettre en place des moyens de gestion de projet afin de traquer l'avancement et résoudre les difficultés rencontrées. Dans ce but, un fichier tableur sur lequel est affiché chacune des tâches non démarrées, en cours, bloquées ou réalisées a été partagé dans un espace cloud interne au projet afin de permettre à tous les membres d'en connaître l'avancement. De plus, des réunions hebdomadaires et bi-mensuelles ont été tenues respectivement avec le superviseur et le tuteur du stage afin de partager le travail fait et orienter celui à venir.

3.1.1 Répartition des tâches

Ce projet a été réalisé en binôme. Afin d'optimiser son avancement, les tâches ont été réparties de manière très spécifique (de courte durée et clairement définies). Le principal avantage retiré de ce mode de fonctionnement est la vue en quasi-totalité du projet par chacun des membres. L'entraide a été un facteur important pour l'aboutissement de ce travail : il n'aurait pu être réalisé de façon individuelle sur la période proposée. En revanche, les sujets abordés dans la suite de ce rapport sont issus d'un travail personnel sur le projet choisi.

De manière plus générale, Tom Berland a joué un rôle d'éclaireur, recherche documentaire sur le déploiement de chacun des composants, installation de ceux-ci sur les différentes machines virtuelles et écriture des premiers playbook ansible en plus d'effectuer la partie d'intégration et administration des comptes. En contrepartie, mon rôle a été l'intégration des différents composants au sein d'un même environnement, vérification du bon fonctionnement des communications et usages, mise en place du contexte propice (certificats, serveur NTP et DNS, jointure au domaine, durcissement des composants, etc).

3.2 Exigences du projet

Le sujet de ce stage a donc été l'automatisation au moyen d'Ansible du déploiement complet de CyberArk et de son environnement. Néanmoins, de nombreuses exigences restent à définir : un premier échange avec notre superviseur de stage Pierre-Antoine Jourdon a permis de poser sur papier les différentes contraintes par degré d'importance et certaines idées d'améliorations en fonction de l'avancement du projet.

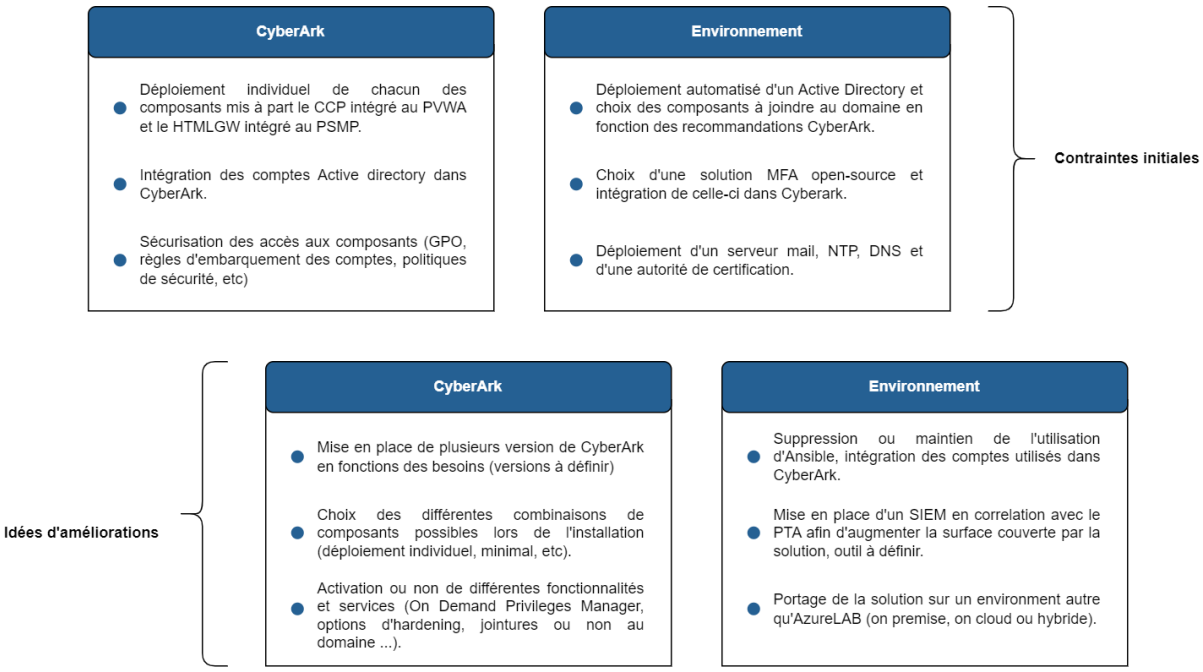


FIGURE 6 – Exigences du sujet

3.3 CyberArk

Solution phare du marché dans la gestion d'accès à privilèges depuis de nombreuses années et, bien que la technologie soit vieillissante et rapidement rattrapée par la concurrence, elle reste aujourd'hui la solution la plus utilisée sur les environnements clients. Composée à partir d'une dizaine de briques plus ou moins essentielles, c'est un des outils PAM les plus complexes à mettre en place. Dans un contexte de formation, un déploiement automatisé est un besoin relativement intéressant afin de permettre aux étudiants de prendre en main la solution rapidement. Il est présenté dans les paragraphes suivants chacun des composants nécessaires au bon fonctionnement et déploiement de la solution ainsi que l'établissement d'une connexion utilisateur.

3.3.1 Fonctionnement et établissement d'une connexion privilégiée

Pour une entreprise dans laquelle la solution CyberArk Privileged Access Security a pu être déployée, un utilisateur ayant des droits administrateurs et souhaitant effectuer une action quelconque sur un serveur d'application doit suivre les étapes décrites ci-dessous :

1. L'utilisateur requête le portail web (PVWA) et s'y authentifie. Une fois connecté, l'ensemble des comptes auxquels il peut avoir accès lui apparaît. L'utilisateur effectue une demande d'accès pour l'un de ces comptes.
2. Si autorisé, le portail web redirige l'utilisateur vers le serveur de rebond (PSM ou PSMP) afin d'établir une connexion vers le serveur d'application.
3. Une connexion est établie entre le client et le serveur de rebond (Protocole RDP ou SSH).
4. Le serveur de rebond fait la demande du secret au Vault (mot de passe ou clé privée).
5. Le serveur de rebond établit une connexion via un protocole natif en direction du serveur cible requêté.
6. la session est enregistrée par le serveur de rebond et téléversée sur le Vault pour une durée définie par les politiques de sécurité CyberArk choisies.
7. Les comportements de chaque utilisateur sont analysés par le PTA, toute action irrégulière étant reportée à un compte administrateur CyberArk.

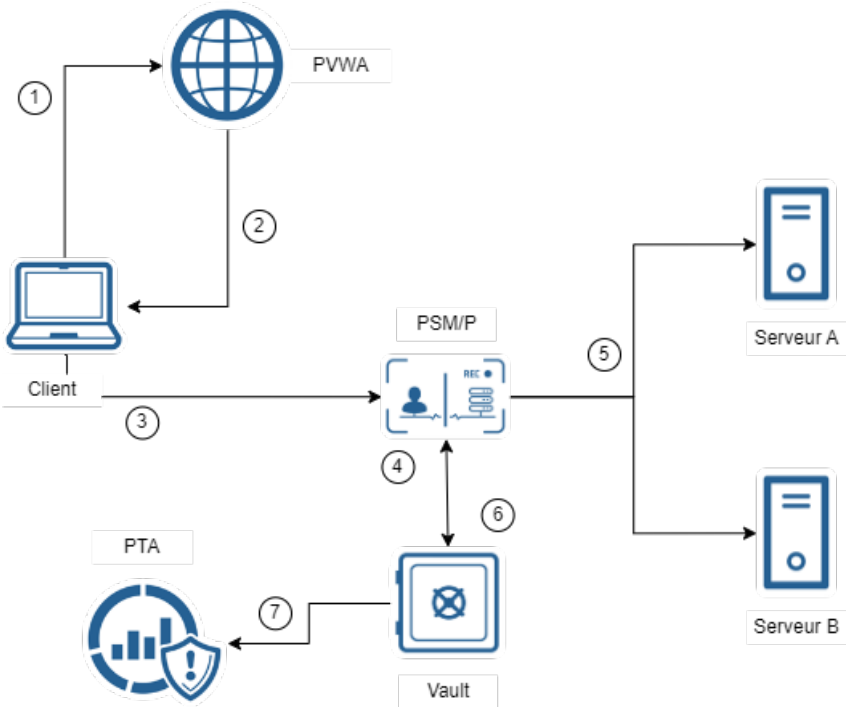


FIGURE 7 – Établissement d'une connexion à un compte privilégié sous CyberArk

3.3.2 Environnement de déploiement

Dans le cadre de ce projet, l'environnement AzuleLAB fourni est composé de 12 machines vierges nommées par le ou les composants devant être installés sur ces dernières. La machine **IaC Workstation** est la station de travail centrale chargée de lancer le déploiement sur toutes les autres, aucun composant n'est installé sur celle-ci mis à part Ansible.

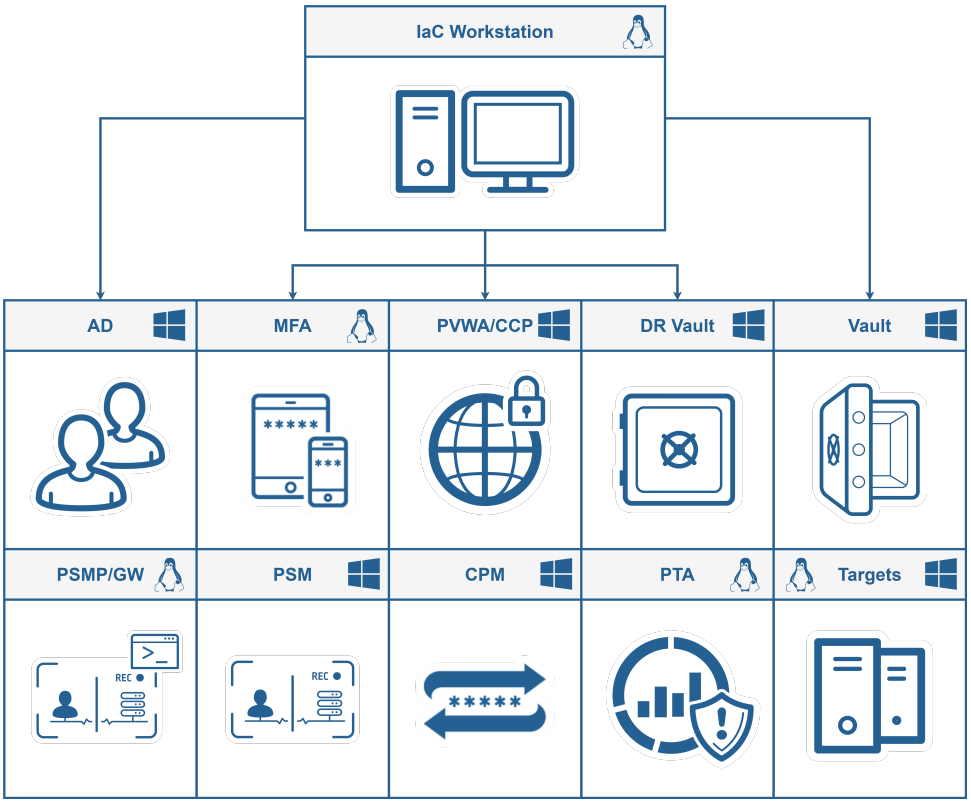


FIGURE 8 – Maquette de l'environnemnt AzureLAB

Les quatre composants centraux de toute solution PAM apparaissent clairement :

— **Coffre fort :** Vault et Disaster Recovery Vault, enregistrement et synchronisation des secrets afin de permettre une disponibilité constante du service.

— **Gestionnaire d'accès :** Password Vault Web Access, interface web de gestion de la solution par les utilisateurs et administrateurs CyberArk.

— **Gestionnaire de session :** Privileged Session Manager (et Proxy), récupération des identifiants et établissement de la connexion pour les utilisateurs Windows (PSM) et Linux (PSMP).

— **Gestionnaire de mots de passe :** Central Policy Manager, rotation des mots de passe et application des politiques de sécurité.

Sont ajoutés à ces composants primaires le Privileged Threat Analytics afin d'effectuer une analyse des comportements et report d'alertes ainsi que le Central Credential Provider permettant la récupération de secret depuis le Vault.

Enfin, afin d'imiter au mieux un environnement client, un Active Directory et comptes utilisateurs associés seront joints à l'environnement en plus d'un serveur MFA à intégrer à la solution.

3.4 Ansible

Parmi les outils d'infrastructure as code, Ansible est un moyen efficace d'effectuer les tâches d'installation et de configuration de services numériques. A l'aide de ses nombreux modules développés par RedHat et la communauté, il est possible de presque tout automatiser depuis une machine distante. Rédigés en YAML, les playbooks exploitent le langage python ainsi que des scripts shell et web services afin d'exécuter chacune des actions requises.

A noter : Python n'est pas nécessaire sur les machines Windows, la plupart des modules Linux étant incompatibles, ceux-ci ont été réécrit en powershell.

3.4.1 Prérequis

Afin de permettre le déploiement automatique des différents composants CyberArk, il est nécessaire de posséder un compte ayant les droits super-utilisateur (prérequis d'Ansible) ainsi que d'un protocole de contrôle à distance pré-installé sur chacune des machines. Sous Linux, SSH est un standard disponible sur Debian et Red Hat entreprise, permettant une connexion facile depuis le noeud central. En revanche, sous Windows, une configuration est nécessaire que ce soit pour l'installation et l'utilisation de open-ssh ou de WinRM. Dans un souci de conformité, le choix a été fait d'utiliser Open-ssh.

Les comptes administrateurs étant déja existants sur toutes les machines, le premier prérequis a été l'installation de Open-ssh serveur sur les systèmes d'exploitation Windows. L'ensemble des machines AzureLAB doit être distribué avec le service sshd actif et prêt à utilisation.

3.4.2 Cahiers de jeu, rôles et tâches

Ansible fonctionne à l'aide de plusieurs éléments ordonnés hiérarchiquement. En premier lieu se situe le cahier de jeu (Ansible Playbook), appelé lors du lancement d'une commande Ansible, il n'agit que comme un conteneur de différent jeux. Au stade inférieur, les jeux (Ansible Play) sont des ensembles d'actions à exécuter sur un ou plusieurs hôtes spécifiés via la variable host. Enfin, les tâches (Ansible Task) sont les éléments en bas de la hiérarchie qui font appel aux modules python ou powershell dans l'objectif d'effectuer une action définie. Par défaut, les tâches sont exécutées de manière séquentielle, la parallélisation n'intervenant qu'au niveau des hôtes.

En dernier lieu, les rôles sont un des composants les plus importants de l'architecture Ansible (Ansible Role). Un rôle peut être vu comme une librairie, c'est-à-dire un ensemble de tâches codépendantes généralement importées et utilisées en boite noire.

Les cahiers de jeu présentés dans la suite de ce rapport sont répartis sous forme de rôles afin de gagner en modularité. Chaque composant à déployer possède un rôle associé (active_directory, secure_shell, Vault, etc)

A noter : La plupart des tâches Ansible sont différentes selon le système d'exploitation sur lesquelles elles sont jouées. Afin de permettre la répartition correcte de celles-ci, le paramètre Ansible **gather_facts** va permettre de récupérer les informations système nécessaires de chacun des hôtes. Ainsi, un fichier de tâches ou de variables pourra être inclus (du plus spécifique au moins spécifique, eg : Ubuntu22.04 $>$ Ubuntu22 $>$ Ubuntu $>$ Linux $>$ Default) sans besoin de vérifier sous quel type d'hôte le playbook est en train d'être joué. Une approche similaire à celle présentée ci-dessous sera adoptée dans la suite du projet :

```yaml
---
- name: Run some task file on hosts with different operating systems
  include_tasks: "{{ file_name }}"
  with_first_found:
    - files:
        - "{{ ansible_distribution|lower }}-{{ ansible_distribution_version|lower }}.yml"
        - "{{ ansible_distribution|lower }}-{{ ansible_distribution_major_version|lower
        ↪  }}.yml"
        - "{{ ansible_distribution|lower }}.yml"
        - "{{ ansible_os_family|lower }}.yml"
        - "{{ ansible_system|lower }}.yml"
        - "defaults.yml"
      paths:
        - "operating_system"
  loop_control:
    loop_var: file_name
```

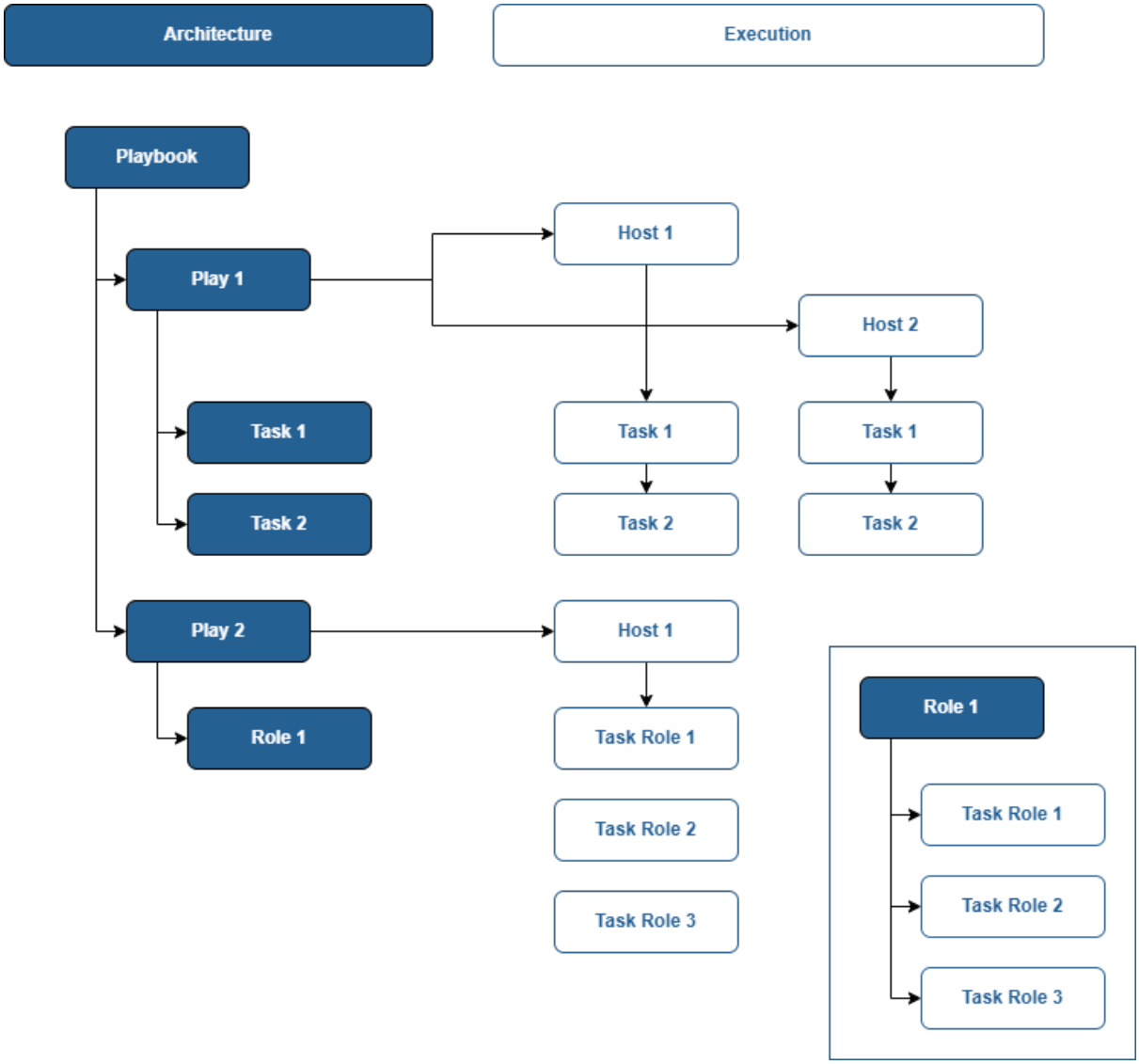


FIGURE 9 – Exécution d'un playbook Ansible

3.4.3 Arborescence du projet

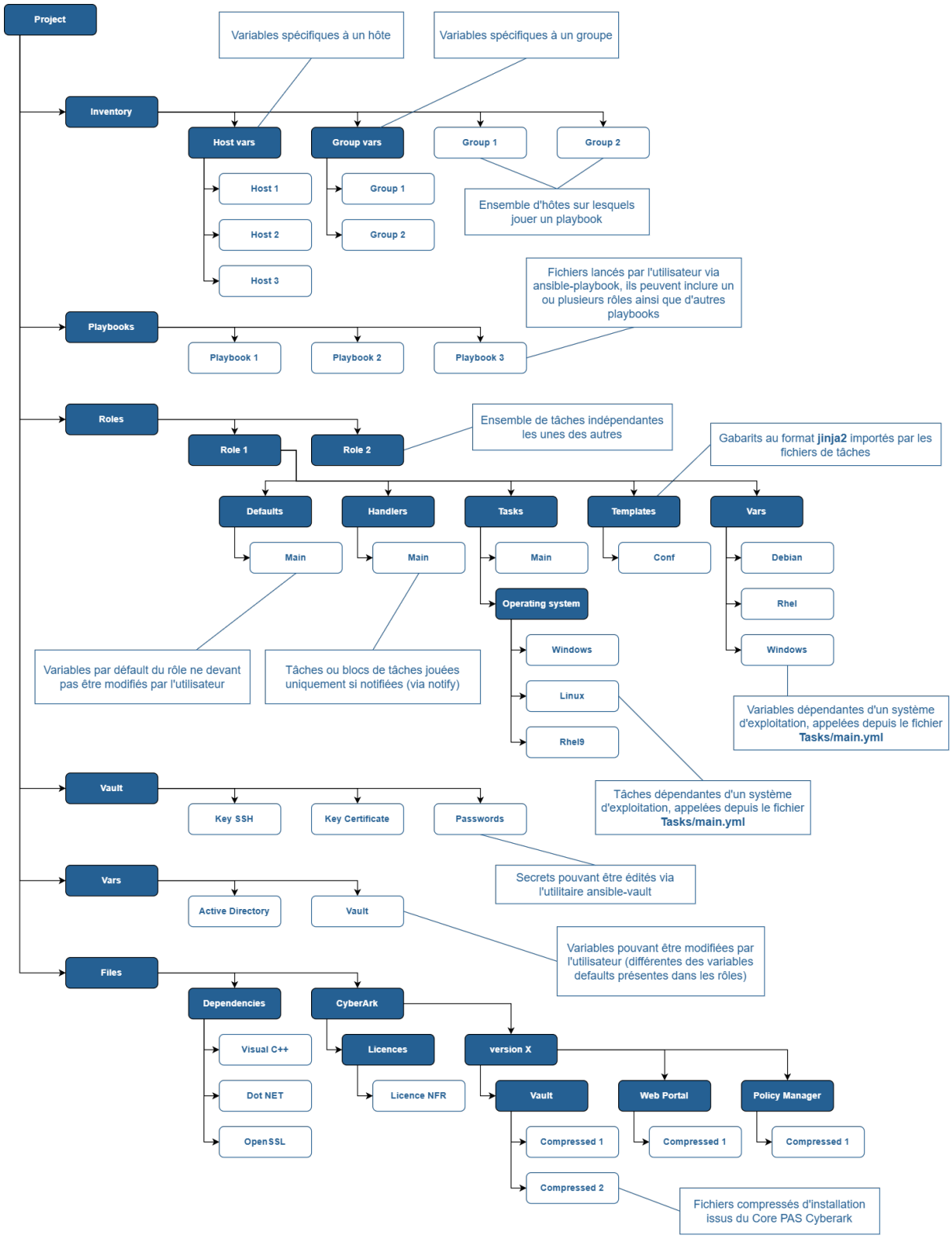


FIGURE 10 – Arborescence du projet

3.5 Environnement de déploiement

3.5.1 Echange de clés SSH

Dans l'idée de garder un niveau de sécurité optimal, la seconde étape (après avoir établi la connexion de l'IaC Workstation aux noeuds contrôlés) a été de mettre en place une connexion via des clés SSH. Le mot de passe du compte super-utilisateur est nécessaire lors du lancement du premier playbook, celui-ci permettant le déploiement de la clé publique maître sur toutes les hôtes Windows et Linux. les connexions suivantes s'effectueront à l'aide de la clé privée générée à cet effet.

Le mot de passe des comptes super-utilisateurs (defender) est le même sur toutes les machines, celui-ci est chiffré en AES-256 via l'utilitaire **Ansible-vault**. Tous les secrets sont enregistrés dans le dossier **/vault**. Pour toute clé privée ayant déjà été générée et importée, une nouvelle clé ne sera pas créée avant la suppression de cette dernière permettant l'exécution idempotente du playbook.

Le playbook d'échange des clés peut être lancé de la manière suivante et sera exécuté sur toutes les hôtes :

```
# Génération de la clé SSH et déploiement de celle-ci sur les machines cibles
ansible-playbook playbooks/secure_shell.yml --ask-vault-pass
```

3.5.2 Infrastructure à clés publiques

Afin de se laisser la possibilité d'utiliser par la suite le protocole WinRM mais aussi d'éviter les avertissements de sécurité du navigateur web, il est intéressant de mettre en place une autorité de certification afin de distribuer des certificats aux différents serveurs le nécessitant.

Lancer le playbook **certificates.yml** :

```
# Déploiement du certificat racine et certificats serveurs
ansible-playbook playbooks/certificates.yml
```

Le playbook est divisé en quatre rôles subsidiaires : le premier consiste à générer une clé permettant de créer le certificat racine auto-signé ; le second partage le certificat racine en tant qu'autorité dans les magasins de certificats des hôtes ; le troisième, quant à lui, génère les différentes clés et certificats serveur et signe ceux-ci avec la clé de l'autorité de certification ; enfin le dernier est chargé d'ajouter le certificat serveur venant d'être généré sur le serveur web Microsoft IIS.

Dans le cadre de ce projet, le certificat racine et clé privée associée sont stockés sur la station Ansible, trois certificats serveurs ont été générés afin de chiffrer les communications vers : le portail web ; la passerelle HTML5 ; le contrôleur de domaine. Les clés privées serveur sont respectivement stockées sur les machines concernées et supprimées de la station Ansible. Pour des raisons de simplicité dans un environnement d'apprentissage, aucun chiffrement n'a été appliqué aux certificats et clés exportés. En revanche tout a été mis en place afin de le permettre au besoin.

Exemple : Génération du CSR pour le certificat racine dans le rôle **Generate_root_certificate** :

```
- name: Create certificate signing request (CSR) for CA certificate
  community.crypto.openssl_csr:
    privatekey_path: ../vault/root_certificate_authority.key
    path: ../certs/root_certificate_authority.csr
    #privatekey_passphrase: "{{ root_certificate_authority_key_passphrase }}"
    common_name: "{{ common_name }}"
    use_common_name_for_san: false  # since we do not specify SANs, don't use CN as a SAN
    basic_constraints:
      - 'CA:TRUE'
    basic_constraints_critical: true
    key_usage:
      - keyCertSign
    key_usage_critical: true
```

La génération d'une infrastructure à clés publiques est un des blocs les plus importants du projet puisque source des connexions chiffrées inter-composants. Elle a représenté un défi de developpement conséquent notammment sur le système d'exploitation de Microsoft au vu du manque de modules Ansible à disposition. Une solution alternative envisagée a été l'installation d'openssl conjugué à l'utilisation du module powershell. Mais utiliser ce genre de module nuit aux principaux avantages apportés par Ansible. La proposition n'a pas été retenue et le choix a été fait de générer sur la station centrale les clés serveur et certificats obtenus afin de les répartir respectivement sur les hôtes.

3.5.3 Annuaire

Un Active Directory Windows a dû être déployé en amont de l'installation de CyberArk. Après étude de celui-ci, il a été défini comme présenté ci-dessous. Plusieurs machines et utilisateurs ont été ajoutés afin de répliquer au mieux un contexte réel. Pour des raisons de sécurité, le Vault n'a pas été ajouté au domaine contrairement au CPM, PSM et PVWA ces derniers étant mois sensibles. Il est important de noter que les sortir du domaine est aussi une solution envisageable privilégiée toutefois par des entreprises à très grand niveau de risque.

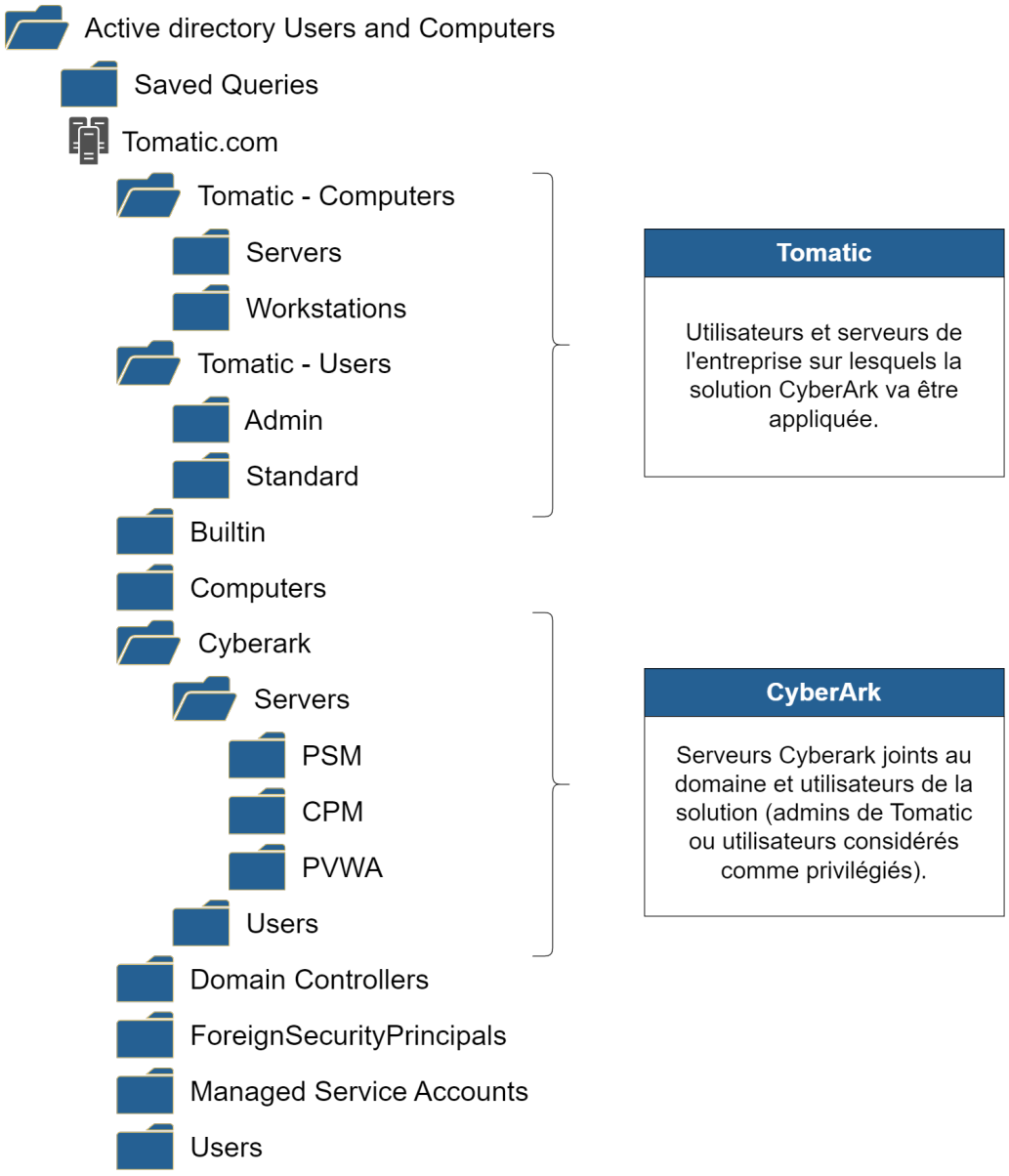


FIGURE 11 – Schéma de l'Active Directory

Le schéma défini n'est pas inaltérable, le fichier de variable **vars/active_directory.yml** contient la définition du nom de domaine, des unités d'organisation, groupes et utilisateurs. Ces variables sont utilisées afin d'instancier l'Active Directory lors de l'étape d'installation du schéma. Concernant les machines à joindre au domaine, le chemin d'enregistrement est à préciser dans le fichier de variable des hôtes concernés.

Le role active directory est divisé en plusieurs parties, l'initialisation du Primary Domain Controller (PDC), l'instatiation du schéma (unités d'orgatisation, groupes et utilisateur) ainsi que la jointure au domaine des machines requises (PVWA, CPM, PSM).

```
# Déploiement de l'AD, ajout des groupes/utilisateurs et jointure au domaine des machines Windows
ansible-playbook playbooks/active_directory.yml
```

A noter : Avant de déployer l'Active Directory, il est nécessaire de renommer chacun des hôtes afin de pouvoir les joindre au domaine (collision de noms dans le cas contraire). Le playbook **prerequisites.yml**, lancé à la suite de l'échange de clés est chargé d'effectuer le changement de nom en plus de la vérification de certaines dépendances (modules python sur les machines Linux entre autres).

3.5.4 Serveur NTP

Un serveur NTP a été installé sur le contrôleur de domaine afin de synchroniser les deux Vaults, étape nécessaire à leur bon fonctionnement. Le premier rôle appelé serveur permet, par l'édition des registres Windows, la définition de la machine Active Directory en tant qu'autorité de temps. Le second rôle, client, va ajouter le contrôleur de domaine en tant que serveur NTP et mettre à jour la configuration afin que celle-ci prenne effet immédiatement.

Le role **network_time_protocol** fait partie d'un ensemble de prérequis lancés via la commande suivante :

```
# Déploiement de l'AD, ajout des groupes/utilisateurs et jointure au domaine des machines Windows
ansible-playbook playbooks/prerequisites.yml
```

3.6 Composants CyberArk

3.6.1 Password Vault

Le Vault est l'élément central de la solution CyberArk, chargé de stocker les secrets (mots de passe et/ou clés) des comptes à privilèges et traçant tout type d'activité qui s'y déroule. Il peut être considéré comme le coeur de l'architecture auquel chaque composant autorisé et qui en a le besoin peut faire appel ; le Vault n'étant pas accessible aux utilisateurs ou à une machine extérieure à l'infrastructure CyberArk.

Au vu de sa criticité, une grande attention doit être accordée lors de son installation afin de répliquer au mieux un environnement de production. D'ordinaire mis en place manuellement, automatiser ce processus a été l'une des tâches les plus conséquentes du projet.

Private Ark Server

Le Private Ark Server est le service Windows installé sur le Vault nécessaire à son bon fonctionnement, principalement en charge d'effectuer les changements de mot de passe (requis par le CPM), de rendre le Vault accessible aux autres composants ainsi que d'envoyer les alertes mail.

Private Ark Client

Le Private Ark Client est une interface Windows permettant d'administrer le Vault. Elle est installée dans un premier temps sur celui-ci afin d'effectuer les premières tâches administratives puis sur des machines d'administration CyberArk nécessitant un accès direct au Vault.

Déployer le Vault signifie mettre en place les deux composants présentés ci-dessus ainsi que le composant annexe Disaster Recovery sur le Vault secondaire. La première étape de l'installation pour chacun des composants consiste à récupérer auprès du magasin CyberArk l'ensemble du dossier **Core Privileged Access Security** contenant tous les scripts et fichiers nécessaires au déploiement complet de la solution, amassant au total 11 giga-octets d'information qui vont être disséminés sur chacune des machines virtuelles correspondantes.

Le fichier de tâche principal, chargé de faire appel à tous les autres, est construit de la façon suivante (similaire sur chaque rôle) :

```yaml
---
- name: Vault - Vault block
  when: vault_install
  block:
    - name: Vault block - Install dependencies
      include_tasks: vault_dependencies.yml

    - name: Vault block - Extract vault package
      include_tasks: vault_extract.yml

    - name: Vault block - Set configuration prerequisites
      include_tasks: vault_prerequisites.yml

    - name : Vault block - Reboot
      ansible.windows.win_reboot:

    - name: Vault block - Install
      include_tasks: vault_install.yml

    - name: Vault block - Harden
      include_tasks: vault_harden.yml

    - name : Vault block - Reboot
      ansible.windows.win_reboot:

- name: Vault - Disaster recovery block
  when: vault_disaster_recovery and disaster_recovery_host
  block:
    - name: Vault disaster recovery block - Disaster Recovery
      include_tasks: vault_disaster_recovery.yml

    - name : Vault disaster recovery block - Reboot
      ansible.windows.win_reboot:
```

En fonction des variables fournies en entrée par le playbook principal, l'installation classique et / ou disaster recovery mode peut être pilotée à distance selon les besoins à chaque appel du rôle. Cela s'est avéré être utile étant donné qu'une dépendance triangulaire existe entre le Vault primaire, l'API web et le Vault secondaire. Ensuite, chacun des fichiers de tâches du rôle est chargé puis exécuté sequentiellement.

La principale difficulté rencontrée lors de l'étape d'installation a été celle de durcissement du serveur. la raison étant l'environnement AzureLAB et son fonctionnement légèrement différent ayant mené à de nombreux ralentissements. Afin de comprendre quels ont été les problèmes rencontrés il est nécessaire de saisir la façon dont AzureLAB effectue le démarrage d'une machine virtuelle :

Attention : la documentation technique d'AzureLAB étant relativement pauvre, le schéma suivant est issu de la compréhension, après tâtonnements, du protocole interne utilisé.

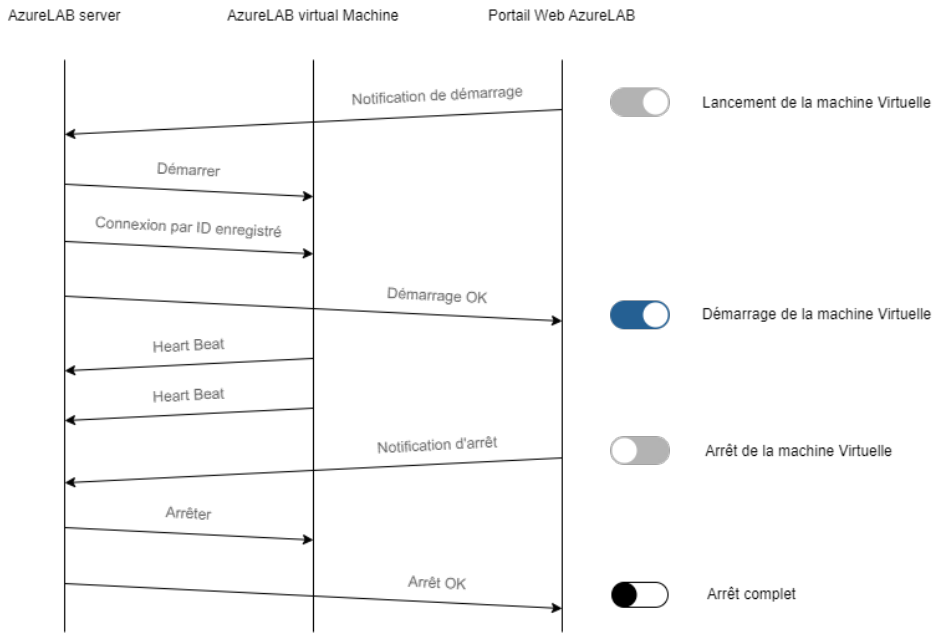


FIGURE 12 – Démarrage et arrêt d'une machine virtuelle AzureLAB

Le trafic réseau intéressant dans le schéma présenté ci-dessus est la première connexion par identifiants enregistrés et les envois appelés **heartbeat**. Le premier permet au serveur de s'assurer du bon démarrage de la machine virtuelle par une connexion à l'aide des mots de passe choisis par l'utilisateur lors de l'instanciation de celle-ci, le second, quant à lui, est un signal régulier envoyé par la machine virtuelle afin de permettre au serveur de connaître son statut.

Refus de la connexion des identifiants enregistrés

Première conséquence de l'étape de durcissement, le Vault active l'option de sécurité windows **"Always Prompt for Password"** empêchant ainsi le serveur AzureLAB de s'authentifier auprès de la machine et renvoyer son état au portail de contrôle web. De ce fait, une machine durcie ne pourra plus être allumée et sera inutilisable une fois éteinte. Par conséquent, Il est nécessaire de modifier les registres Windows afin de ré-établir le paramètre avant le rallumage de la machine virtuelle via AzureLAB.

```
---
- name: Vault Harden - Group policies, disable always prompt for password (AzureLAB)
  ansible.windows.win_regedit:
    path: HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services
    name: fPromptForPassword
    data: 0
    type: dword
  notify:
    - Vault handler - Update registries

```

Deconnexion de la station Ansible

Lors de l'étape de durcissement, de nombreuses parties du système sont renforcées dont le trafic réseau. Toutes les règles pare-feu sont supprimées isolant ainsi le Vault de tout contact entrant ou sortant. Les seules communications autorisées sont le port TCP 1858 permettant un lien direct au Vault des composants CyberArk (CyberArk Vault protocol) ainsi que le trafic RDP sur le port 3389 si autorisé lors de l'installation. La nécessité de pouvoir accéder au Vault au moyen d'autres protocoles sur des ports différents est un cas d'usage commun et pris en compte par CyberArk, modifier le fichier **dbparm.ini** permet au Vault d'ajouter des règles pare-feu lors du redémarrage du service **PrivateArk**.

Fichier dbparm.ini et règles firewall :

```
...

[FIREWALL]
AllowNonStandardFWAddresses=[{{ client_ip_range }}],Yes,3389:outbound/udp,3389:inbound/udp
AllowNonStandardFWAddresses=[{{ client_ip_range }}],Yes,3389:outbound/tcp,3389:inbound/tcp
AllowNonStandardFWAddresses=[{{ ansible_workstation }}],Yes,22:outbound/tcp,22:inbound/tcp

```

En revanche, comme précisé ci-dessus, les règles ne sont prises en compte que lors du redémarrage du service. Pendant l'étape de durcissement, la station Ansible est donc déconnectée des hôtes tant qu'un redémarrage manuel du Vault (ou du service) n'a pas été fait. Afin de contourner ce problème, la solution a été d'ajouter des règles pare-feu temporaires (écrasées par le service **PrivateArk** lors du redémarrage) après l'exécution de **CAVaultHarden.exe**. Le modèle ci dessous permet l'ajout des règles suite au durcissement sans laisser le temps à Ansible de perdre la main :

```
# Run hardening script
& "{{ private_ark_folder }}\Server\Hardening\CAVaultHarden.exe" StandardVault

# Add firewall rules (note that they will be erased on reboot)
& New-NetFirewallRule -DisplayName TCP-SSH-Vault-Access -Protocol TCP -LocalPort 22
↪  -RemotePort Any -Direction Inbound -RemoteAddress {{ ansible_workstation }} -Action
↪  Allow -Profile Any -Group CYBERARK_RULE_NON_STD_ADDRESS

...
```

Suppression du Heartbeat

Une fois les étapes précédentes effectuées, le Vault peut redémarrer et la connexion Ansible est maintenue. Toutefois, après une quinzaine de minutes les machines sont éteintes par AzureLAB pour la raison suivante : "La machine virtuelle x semble inactive, une procédure d'arrêt forcé a été engagée afin d'éviter une surfacturation". Cette erreur vient du fait que les paquets **heartbeat** envoyés par la machine virtuelle vers le serveur AzureLAB permettent à ce dernier de connaître l'état de la machine et forcer l'arrêt si aucun trafic n'est reçu.

Deux solutions permettent de remédier à ce problème :

— Analyser le trafic réseau sortant et autoriser en conséquence les protocoles et ports utilisés par AzureLAB dans l'objectif de garder la machine en vie.

— Changer la procédure de durcissement et ne pas effectuer la partie trafic réseau.

Le Vault est le premier composant à être déployé et son automatisation chronophage. Bien que difficile, le choix a été fait de partir sur la seconde solution afin d'avancer au mieux sur la suite du projet. De plus, ces problèmes étant des spécificités d'AzureLAB, il n'est pas intéressant de s'y attarder. L'option a toutefois été laissée de revenir sur ce composant à la fin du projet.

Vault Disaster Recovery Module

Jusqu'ici l'installation du Vault Disaster Recovery est relativement similaire. La seule différence notable étant l'utilisation des mêmes clés générées lors de l'installation du Vault par l'utilitaire **PrivateArk Key Gen** en plus d'importer un paquet supplémentaire lors de l'étape de téléversement et d'extraction.

Pour être en mesure d'installer le Vault, le PVWA doit avoir été mis en place puisqu'un appel à l'API CyberArk est nécessaire pour activer et réinitialiser le mot de passe de l'utilisateur DR. Il est donc, dans un premier temps, requis de récupérer un jeton d'authentification au moyen d'identifiants CyberArk :

```yaml
- name: Vault DR - Fetch logon token
  ansible.builtin.uri:
    url: "https://{{ pvwa_domain_name
    ↪  }}/PasswordVault/WebServices/auth/Cyberark/CyberArkAuthenticationService.svc/Logon"
    method: POST
    body_format: json
    body:
      username: "{{ vault_username }}"
      password: "{{ vault_password }}"
    return_content: yes
    validate_certs: true
  register: response
  delegate_to: localhost
  run_once: true
```

Le jeton ayant été récupéré dans la variable **response**, il peut être utilisé en vue d'effectuer les modifications requises :

```yaml
- name: Vault DR - Enable DR User
  ansible.builtin.uri:
    url: "https://{{ pvwa_domain_name }}/PasswordVault/API/Users/{{ DR_user_id }}/enable/"
    method: POST
    headers:
      Authorization: "{{ response.json | community.general.json_query('CyberArkLogonResult')
      ↪  }}"
    validate_certs: true
  delegate_to: localhost
  run_once: true

- name: Vault DR - Update DR user password
  ansible.builtin.uri:
    url: "https://{{ pvwa_domain_name }}/PasswordVault/API/Users/{{ DR_user_id
    ↪  }}/ResetPassword/"
    method: POST
    headers:
      Authorization: "{{ response.json | community.general.json_query('CyberArkLogonResult')
      ↪  }}"
    body_format: json
    body:
      id: "{{ DR_user_id }}"
      newPassword: "{{ DR_password }}"
    validate_certs: true
  delegate_to: localhost
  run_once: true
```

L'exécutable présent dans le paquet Disaster Recovery Module peut alors être lancé dans le but de répliquer les safes du Vault et se tenir prêt en cas de malfonctionnement de la solution.

```yaml
- name: Vault DR - Run disaster recovery setup
  ansible.windows.win_shell: "{{ disaster_recovery_component_folder }}\\setup.exe /s /f1\"{{
  ↪  disaster_recovery_component_folder }}\\vault_DR.iss\""
  delegate_to: vault_DR
  run_once: true
```

Lancer l'installation du Vault s'effectue de la manière suivante :

```
# installation des deux vaults et vault DR si spécifié
ansible-playbook playbooks/vault.yml --ask-vault-pass
```

3.6.2 Password Vault Web Access

Le Password Vault Web Access abrége PVWA ou portail web est un site internet déployé sur un serveur Microsoft IIS constituant le point d'entrée par lequel chaque utilisateur doit passer avant de pouvoir établir une nouvelle connexion, administrer les politiques de sécurité ainsi que les comptes et coffres déployés ou encore monitorer l'activité des autres utilisateurs. Contrairement au Vault, il existe des rôles Ansible développés par CyberArk permettant de déployer le PVWA, le CPM ainsi que le PSM. Ces rôles, bien que peu mis à jour et ne supportant pas la version actuelle de CyberArk, ont été d'une grande utilité afin d'accélérer le processus de développement.

La principale difficulté liée au déploiement de ce composant a été la mise en place de l'infrastructure à clés publiques ainsi que la vérification de son bon fonctionnement puisque c'est le premier composant nécessitant un certificat serveur. Une fois celui-ci généré et déployé au format pksc12 sur le PVWA à l'aide du playbook **certificates.yml** présenté plus haut, il est nécessaire de l'importer dans le magasin de certificats Windows (en plus du certificat racine en tant qu'autorité de certification) puis de l'attacher au portail web depuis l'interface de configuration du serveur IIS.

Le certificat déplacé sur le serveur, il peut être enregistré de la manière suivante :

```yaml
- name: Sign server certificate block - import certificate
  ansible.windows.win_certificate_store:
    path: "{{ win_remote_certs }}\\server_certificate.pfx"
    store_name: My
    file_type: pkcs12
    store_location: localMachine
    key_exportable: true
    key_storage: machine
    state: present
```

La variable **store_name** permet de spécifier le store dans lequel enregistrer le certificat, **My** signifie le store personnel de la machine locale ou de l'utilisateur courant conformément a **key_storage**. Cette même variable instanciée avec la valeur **root** permet l'installation du certificat en tant qu'autorité de certification et ainsi la possibilité de vérifier ceux ayant été déployés via cette autorité.

Une fois enregistré, l'empreinte du certificat fraîchement installé peut être obtenue en récupérant et filtrant tous les certificats présents dans le magasin personnel Windows afin de l'attacher au serveur à l'aide du module Ansible **win_iis_webbinding** :

```yaml
- name: Deploy certificates - Obtain information on each certificate that was touched
  community.windows.win_certificate_info:
  register: certificates_stats

- name: Deploy certificates - Set certificate thumbprint
  ansible.builtin.set_fact:
    thumbprint: "{{ item.thumbprint }}"
  when: item.subject == 'CN={{ hostname }}' and item.friendly_name == '{{ hostname }}
  ↪  certificate'
  loop: "{{ certificates_stats.certificates }}"

- name: Deploy certificates - bind it to website
  community.windows.win_iis_webbinding:
    name: "{{ website_name }}"
    certificate_hash: "{{ thumbprint }}"
    protocol: https
    port: 443
```

Le playbook du PVWA peut être lancé de la manière suivante :

```
# installation du PVWA sans certificat
ansible-playbook playbooks/password_vault_web_access.yml --ask-vault-pass
```

Le playbook du PVWA est dépendant de celui du **Vault** (ou d'un Vault déja en place), le déploiement des certificats sera effectué après l'installation à l'aide du rôle **certificates** présenté plus haut dans ce document.

3.6.3 Central Policy Manager

Le Central Policy Manager est le module consacré à la mise en pratique des politiques de sécurité, principalement la vérification, réconciliation et rotation des secrets sur le Vault et les serveurs cible.

L'installation du composant suit les mêmes principes à la différence près que le CPM est joint au domaine et va nécessiter une attention particulière afin de permettre à tous ses services de fonctionner correctement. CyberArk fournit dans les fichier d'installation des GPO spécifiques à appliquer en fonction d'une jointure ou non au domaine ou d'une installation individuelle / compartie.

Dans le cas présent, il s'agit d'une installation isolée (StandAlone) et jointe au domaine, les GPOs devront par conséquent être appliqués sur le contrôleur de domaine :

```
- name: CPM group policy objects - Select correct GPO settings
  ansible.builtin.set_fact:
    cpm_gpo_backup: "CyberArk\ Hardening\ -\ Windows\ In\ Domain\ -\ Standalone\ -\ CPM-V2"
    cpm_gpo_backup_name: CPM_StandAlone
  when: ansible_windows_domain_member

- name: CPM group policy object block - Create, Import and Link new GPO to CPM
↪  Organisationnal Unit
  when:
  ansible.windows.win_shell: |
    New-GPO -Name "{{ cpm_gpo_name }}" -Domain "{{ ad_full_domain_name }}"
    Import-GPO -TargetName "{{ cpm_gpo_name }}" -BackupGpoName "{{ cpm_gpo_backup_name }}"
    ↪  -Path "{{ root_folder }}\\GPOs\\{{ cpm_gpo_backup }}"
    New-GPLink -Name "{{ cpm_gpo_name }}" -Target "{{ domain_path }}"
  delegate_to: domain_controller
  run_once: true
```

Les GPOs importés ci-dessus permettent, parmi d'autres actions, d'octroyer les droits nécessaires aux utilisateurs locaux générés par CyberArk lors de l'étape d'installation. Il permettent ainsi le démarrage de certains services non essentiels mais néanmoins importants.

Le playbook d'installation du CPM peut être lancé de la manière suivante :

```
# installation du CPM
ansible-playbook playbooks/central_policy_manager.yml --ask-vault-pass
```

Attention : afin d'éviter des scénarii dans lesquels le Vault et le Vault Disaster Recovery seraient désynchronisés (split brain scénario) il est important de lier le CPM à un seul Vault à la fois (puis effectuer le changement manuellement ou automatiquement en cas de besoin). Dans le cas contraire, si pour une raison quelconque les deux Vaults venaient à être actifs au même moment, une rupture pourrait apparaître (différents mots de passe sur différents Vaults).

3.6.4 Privileged Session Manager

Le PSM/P communément appelé serveur de rebond, est la plateforme chargée d'établir, gérer et enregistrer les connexions entre un utilisateur et un Serveur cible. Pour ce faire, une requête vers le Vault va être effectuée afin de récupérer les identifiants, ceux-ci ne transitant jamais sur la machine utilisateur.

Le serveur de rebond peut être décliné sous deux formes :

— **PSM :** serveur Windows sur lequel l'utilisateur peut se connecter à l'aide d'un fichier Remote Desktop Protocol (RDP) récupéré en passant par le PVWA.

— **PSMP :** serveur Linux ajouté afin de simplifier l'accès aux administrateurs Linux ne souhaitant pas passer par le portail web ou des fichiers RDP. La connexion s'effectue à l'aide du protocole SSH en spécifiant le compte et le serveur ciblé *(utilisateur@compte@cible@psmp)*

L'Administration du Vault peut être effectuée depuis le PSM via l'interface PrivateArk Client (aussi installée sur le Vault). Le PSM étant lui aussi joint au domaine, il requiert les mêmes étapes que le CPM afin de mettre en place les GPOs corrects.

Malgré un processus d'installation du PSM Proxy complètement différent (système d'exploitation distinct), son déploiement a été relativement facilité par l'étude de la documentation CyberArk.

Bien que la rotation des mots de passe soit fortement conseillée suite à toute installation à l'aide des playbooks présentés, l'effort a tout de même été fait d'utiliser des **Secure-Strings** Windows afin de ne pas laisser un quelconque secret en clair sur des fichiers de registres :

```
- name: PSM register set password block - Convert to securestring
  become: true
  become_method: runas
  become_user: "{{ ansible_user }}"
  ansible.windows.win_shell: |
    $secure_vault_password_object = ConvertTo-SecureString "{{ vault_password }}"
    ↪  -AsPlainText -Force
    $secure_vault_password_string = ConvertFrom-SecureString $secure_vault_password_object
    write-output $secure_vault_password_string
  register: secure_vault_password_string

- name: PSM register set password block - Set variable for secured vault password
  ansible.builtin.set_fact:
    secure_vault_password: "{{ secure_vault_password_string.stdout_lines[0] }}"
    vault_password: ''
```

Les playbooks du PSM et PSMP peuvent être lancés respectivement sur des machines Windows et Linux de la manière suivante :

```
# installation du PSM et enregistrement dans le vault
ansible-playbook playbooks/privileged_session_manager.yml --ask-vault-pass

# installation du PSMP et enregistrement dans le vault
ansible-playbook playbooks/privileged_session_manager_over_ssh.yml --ask-vault-pass
```

3.6.5 HTML5 Gateway

L'HTML5 Gateway est un composant Cyberark permettant à un utilisateur d'ouvrir une connexion vers le PSM au travers d'un websocket sur le port 443 (https). Ainsi, il est possible de se connecter à une machine cible sans passer par du RDP mais uniquement à l'aide d'un navigateur web.

Le composant a été installé à l'aide du gestionnaire de conteneurs Podman. Les prérequis installés et les scripts d'installation exécutés, une instance peut être générée comme suit :

```
- name: HTML gateway instanciate - Create container
  become: true
  ansible.builtin.shell:
    cmd: ./html5_console.sh run -ti -d -p 443:8443 -v {{ root_store_location
    ↪  }}:/opt/import:ro -e AcceptCyberArkEULA=yes -e GWCert={{ remote_certs
    ↪  }}/server_certificate.crt -e GWKey={{ remote_vault }}/server.key -e GWCAFile={{
    ↪  root_store_location }}/root_certificate_authority.crt -e EndPointAddress=https://{{
pvwa_domain_name }}/PasswordVault --name {{ hostname }} cahtml5gw:14_0_0_10
  args:
    chdir: "{{ HTMLGW_component_folder }}/PSMGWDocker"
```

Note : il est important d'avoir exécuté le playbook certificates au préalable étant donné qu'un certificat serveur doit être généré afin de lancer correctement l'instance.

L'installation du HTML5 Gateway peut être lancée de la manière suivante :

```
# installation du HTML5 Gateway
ansible-playbook playbooks/html_gateway.yml --ask-vault-pass
```

3.6.6 Privileged Threat Analytics

Le rôle du PTA est la surveillance en continu de toute activité des comptes à privilèges gérés par la solution CyberArk Privileged Access Manager - Self Hosted. Son mode de fonctionnement est le suivant : un motif est établi en fonction des patrons comportementaux de chaque utilisateur au fil du temps ; tout changement brusque dans ces comportements sera détecté et remonté en tant qu'anomalie ou intrusion dans le système (accès au Vault en heures creuses, utilisateur dormant subitement actif, utilisation d'un compte privilégié non enregistré, etc).

Les données récoltées par le Privileged Threat Analytics peuvent être envoyées à un SIEM afin d'augmenter la qualité des alertes et l'efficacité de la solution. Afin de simplifier son installation, CyberArk propose un rôle Ansible complet permettant de déployer le composant dans sa totalité. Néanmoins, comme pour le PAS Orchestrator, le rôle nécessite certaines modifications avant de pouvoir être utilisé correctement.
Une étape nécessaire à la récupération du jeton d'authentification auprès de l'API du PTA et le redémarrage du serveur **Tomcat** à l'aide de l'utilitaire **run.sh** :

```
- name: Start tomcat server
  become: true
  ansible.builtin.shell: printf '4\n' | /opt/tomcat/utility/run.sh
```

Le jeton peut ainsi être récupéré et la suite de l'installation / configuration aboutie :

```
- name: Fetch Remote token
  ansible.builtin.uri:
    url: "https://{{ remote_machine_ip }}:8443/installer/api/getauthtoken"
    method: POST
    body_format: form-urlencoded
    body:
      username: "{{ ansible_user }}"
      password: "{{ ansible_password }}"
    validate_certs: false
  register: login_attempt
  until: login_attempt.status == 200
  retries: 10
  delay: 5
  when: not ansible_check_mode
```

3.7 Embarquement des comptes et tests d'utilisation

3.7.1 Etape d'onboarding

Dans l'intention de mettre en corrélation les comptes présents dans l'annuaire et l'outil CyberArk, une étape d'embarquement des comptes a été nécessaire afin d'autoriser ces nouveaux comptes ajoutés à se connecter à la solution CyberArk au moyen du protocole LDAP. A cette fin, il est nécessaire de passer à nouveau par l'API CyberArk disponible sur la machine PVWA. Une fois les comptes ajoutés, ils apparaissent dans le Vault après une première connexion.

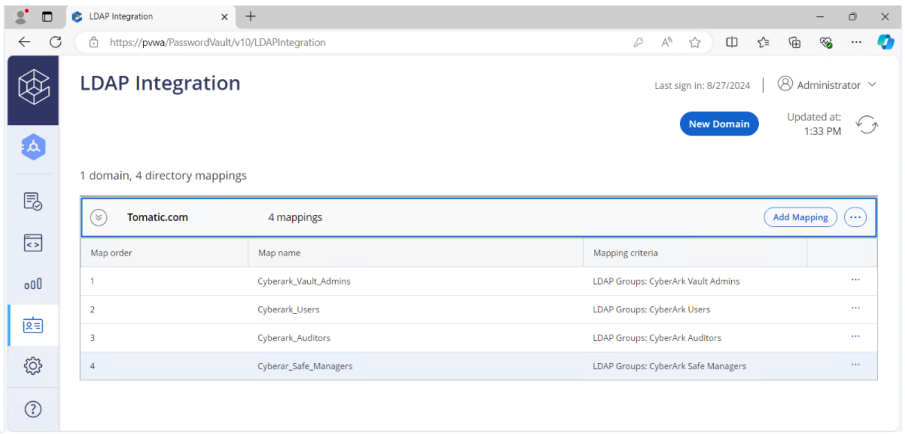


FIGURE 13 – AES - Counter mode

3.7.2 Utilisation de la solution

Une fois la solution entièrement déployée, les comptes administrateurs locaux des machines cibles Windows et Linux ont été intégrés à la solution au travers du rôle **onboarding/local_accounts**. Les Safes CyberArk **WinAdmins** et **LinAdmins** sont crées et les comptes ajoutés à ces derniers. Afin de tester l'ensemble, il est possible de se connecter au PVWA à l'aide d'un compte ayant des droits sur ces Safes et lancer une nouvelle connexion vers la machine cible au travers du PSM/P.

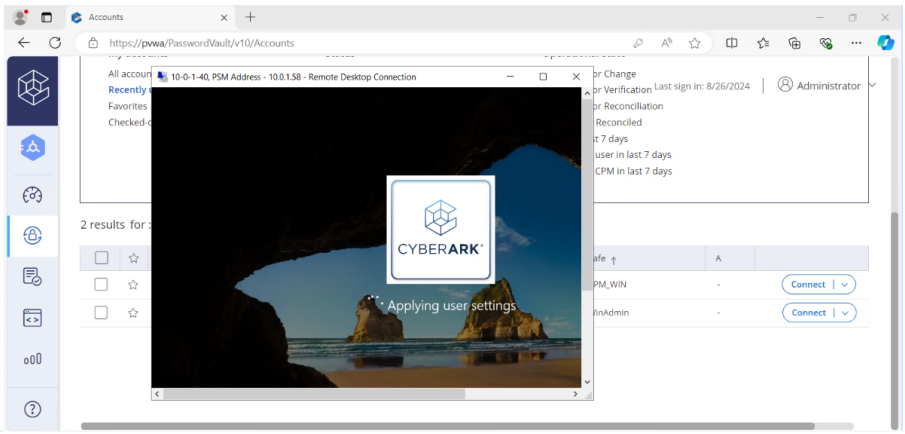


FIGURE 14 – AES - Counter mode

4 Résultats

4.1 Fonctionnalités et composants déployés

Tous les composants CyberArk ont été déployés avec succès et intégrés à la solution, c'est une réussite autant d'un point de vue technique que d'avancement sur les prévisions du projet. Ce travail a permis l'obtention de compétences effectives aussi bien sur l'automatisation du processus de déploiement que sur la compréhension des technologies utilisées. Les composants externes à CyberArk, l'Active Directory ainsi que l'infrastructure a clés publiques sont fonctionnels et modulaires à l'aide des fichiers de variables fournies avec la solution.

4.2 Les concessions

Le projet a été concluant sur la plupart des points discutés lors de son lancement, certaines concessions ont malgré tout dû être faites. Par exemple, une seule et unique version de CyberArk (14) a été déployée et non plusieurs comme envisagé au début du sujet. Ou encore, l'intégration modulaire des composants n'a pas été implémentée (choix d'installation individuelle ou compartie sur les modules le permettant) bien que cette modularité puisse être obtenue avec peu de travail supplémentaire.

Le composant externe MFA a été délégué à des membres en intermission proposant leur aide sur le projet. Il n'a malheureusement pas été finalisé à cause de contretemps personnels ou de priorisation d'autres tâches. Il est toutefois maintenant en bonne voie d'être opérationnel rapidement.

Enfin, l'intégration des comptes Ansible dans CyberArk a été laissée en suspens, cela présente un risque si la solution venait à être déployée dans un contexte réel étant donné que des comptes avec droits super-utilisateurs resteraient actifs sans rotation de mot de passe et surveillance aucune.

4.3 Suite pour le projet

La documentation complète du processus d'utilisation de ce travail fournie à l'équipe PAM de Capgemini est en annexe de ce rapport. Ce projet doit être maintenu afin d'atteindre l'objectif final : un déploiement complet de la solution sur une architecture réelle on premise ou on cloud. Dans ce but, de nombreuses sessions de présentation ainsi que d'utilisation de l'outil ont été faite à plusieurs membres de la communauté PAM et IAM. Les explications apportées lors de ces présentations ont été appréciées par les membres de la communauté.

5 Sujet annexe : Etude de la génération d'IV sur AES-CTR

En plus du projet principal, une mission complémentaire pour un client du secteur de la défense a été demandée : déterminer un algorithme de génération d'Initial Vector (IV) utilisé dans une reconstruction d'AES counter mode. Mettre à défi la ou les solutions apportées et présenter ces dernières aux équipes dédiées a été le travail effectué durant la semaine consacrée à cette tâche.

EGNOS Ranging and Integrity Monitoring Stations V3

Lors du développement de la version 3 des stations RIMS de la société EGNOS, la fonction AES-CTR a été entièrement reconstruite afin d'être supportée par les composants hardware. AES-CTR nécessite un nonce afin de chiffrer chacun des blocs d'un message donné. Un Générateur Congruentiel Linéaire a été proposé afin de générer les IV, ce dernier n'étant pas adapté à une utilisation cryptographique (CSPRNG), la proposition a été rejetée. L'objectif est d'étudier le fonctionnement d'AES-CTR et proposer une fonction de génération d'aléatoire permettant :

— la rotation des clés tous les 5 ans,
— un nombre maximal de messages envoyés établi à 50 par seconde ($\approx 2^{33}$ par clé).

5.1 AES - Counter mode

Advanced Encryption Standard (AES) peut fonctionner sous plusieurs modes, le mode counter (CTR) propose un chiffrement non-dépendant (chaque bloc pouvant être chiffré / déchiffré en parallèle) par blocs de 128 bits.

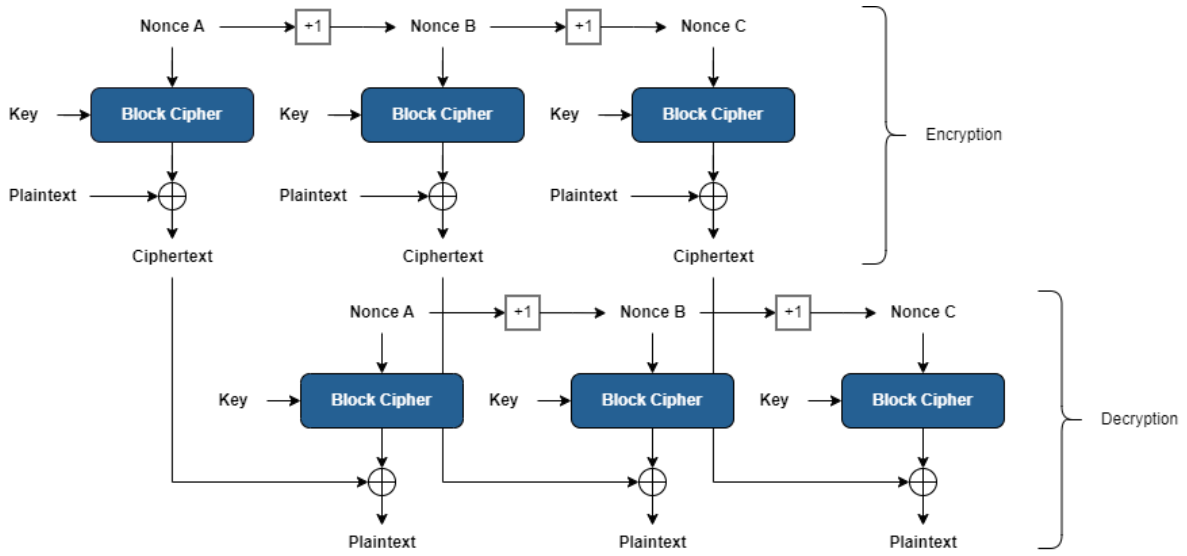


FIGURE 15 – AES - Counter mode

A noter : il est indispensable que les paires (Clé, Nonce) soient uniques, utiliser plusieurs fois le même nonce lors de la durée de vie d'une clé conduirait à la compromission de la confidentialité des messages par une attaque relativement simple. A contrario, la non prédictibilité des nonces n'est pas un facteur important étant donné qu'il n'existe pas d'attaque à texte clair connu / choisi sur AES (mis à part par force brute).

Supposons :

— P_1 et P_2 des textes clairs
— C_1 et C_2 leur correspondant chiffré
— $Nonce_A$ un nombre unique généré aléatoirement
— key la clé AES utilisée

Alors :

$$C_1 = P_1 \oplus AES(key, Nonce_A)$$
$$C_2 = P_2 \oplus AES(key, Nonce_A)$$

$$\begin{aligned} C_1 \oplus C_2 &= P_1 \oplus \cancel{AES(key, Nonce_A)} \oplus P_2 \oplus \cancel{AES(key, Nonce_A)} \\ &= P_1 \oplus P_2 \end{aligned}$$

car $AES(k_1, t_1) = AES(k_2, t_2)$ si $k_1 = k_2$ et $t_1 = t_2$

Donc :

$$P_2 = P_1 \oplus C_1 \oplus C_2$$
$$P_1 = P_2 \oplus C_2 \oplus C_1$$

Une utilisation correcte d'AES-CTR serait :

$$C_1 = P_1 \oplus AES(key, Nonce_A)$$
$$C_2 = P_2 \oplus AES(key, Nonce_B)$$

Avec $Nonce_A$ et $Nonce_B$ différents

5.2 Méthode de création du nonce

Sur la majorité des implémentations d'AES-CTR, le nonce est décomposé de la manière suivante : $< IV_i , Counter >$ avec IV_i le vecteur initial du message M_i et $Counter$ une variable incrémentée pour chaque bloc du message M_i, réinitialisée à chaque nouveau message. La répartition des bits alloués à l'IV et au $Counter$ est laissée au choix de l'utilisateur sur une taille totale de 128 bits (taille d'un bloc).

	M_i	M_{i+1}
$Block_0$	$< IV_i , 1 >$	$< IV_{i+1} , 1 >$
$Block_1$	$< IV_i , 2 >$	$< IV_{i+1} , 2 >$
...	...	...
$Block_n$	$< IV_i , n >$	$< IV_{i+1} , n >$
$Block_{n+1}$	$< IV_i , n+1 >$	$< IV_{i+1} , n+1 >$

En revanche, une telle construction du nonce n'est pas obligatoire ; comme précisé au dessus, le seul prérequis est l'unicité de chaque paire (Clé, Nonce). Nous allons voir dans les sections suivantes différentes manières de les construire.

5.2.1 Compteur et double-compteur

Intuitivement, utiliser un compteur de 2^{128} incrémenté à chaque bloc indépendamment du message sur toute la durée de vie de la clé semble être la solution la plus simple à implémenter. Toutefois, bien que ce procédé ne pose pas de problèmes du point de vue sécurité (sur 5 ans, le nombre de message ne devrait pas dépasser 2^{33}), il érige une complication lors du déploiement. Pour tout bloc perdu ou corrompu, comment s'assurer de l'état du compteur ? Une solution de synchronisation devra être développée afin de pallier à cette difficulté.

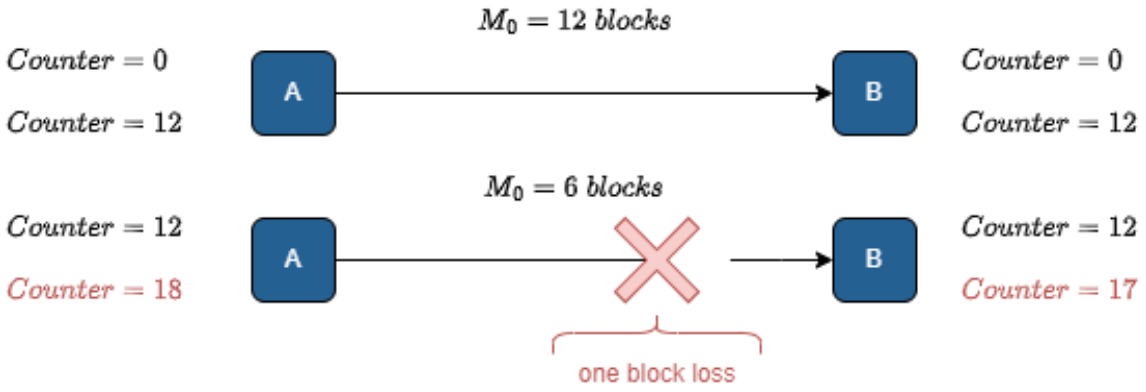


FIGURE 16 – AES - Désynchronisation du compteur

Un double-compteur, décomposé de la manière suivante $< Counter_1, Counter_2 >$, $Counter_1$ incrémenté à chaque nouveau message et $Counter_2$ incrémenté à chaque nouveau bloc (ce dernier réinitialisé à chaque message), est une solution envisageable. Ainsi la perte d'un ou plusieurs blocs n'entraînerait pas la perte du compteur de messages par sa présence sur chacun des blocs. Une solution de synchronisation, plus simple à déployer, doit tout de même être mise en place au niveau des messages.

Algorithm 1 64 bits upper and lower counters

```
upperCounter ← 0
while true do
    lowerCounter ← B                    // B = Number of blocks in current message
    for i ← 0 to lowerCounter do
        N_i ← upperCounter || lowerCounter    // N_i = Nonce used for block i
        lowerCounter++
    end for
    upperCounter++
end while
```

A noter : tout type de compteur ayant une période suffisante peut être utilisé (eg : Registre à décalage à rétroaction linéaire, recommandation du Nist : NIST 800 38 A - Appendix B.1).

5.2.2 Fonctions de hachage

Un autre moyen de générer de l'aléatoire serait d'utiliser des fonctions de hachage. Cependant, Il doit être gardé en mémoire que toute fonction de hachage introduit avec elle une probabilité de collision (cf Paradoxe des anniversaires et Principe des tiroirs). Pour toute fonction de hachage H produisant une sortie de taille n bits, générer $2^{n/2}$ hachés apporte une probabilité de collision de 50%.

D'après les recommandations du NIST (NIST 800 107 R1 - Section 5) il est possible de tronquer une fonction de hachage si les principes suivants sont respectés :

— la taille x du haché tronqué doit être inférieure à la taille de sortie de la fonction de hachage,
— les bits les plus à gauche doivent être gardés, ceux à droite supprimés,
— la taille x du haché tronqué doit être au moins deux fois supérieure au nombre de bits de sécurité recherchés.

Algorithm 2 128 bits hash length

$state \leftarrow seed$
while true **do**
  $counter \leftarrow B$ // B = Number of blocks in current message
  $state \leftarrow H(state)$
  for $i \leftarrow 0$ to $counter$ **do**
    $N_i \leftarrow Trunc_{128}(H(state \parallel i))$ // N_i = Nonce used for block i
  end for
end while

Tronquer la sortie de la fonction de hachage à 128 bits permettrait la génération de 2^{64} hachés avant d'atteindre un risque de collision élevé. L'inconvénient de cette méthode est le besoin constant de garder l'état courant du message actuel (en considérant que ce dernier n'est pas envoyé séparément avec chaque message).

Algorithm 3 64 bits hash length

$state \leftarrow seed$
while true **do**
  $counter \leftarrow B$ // B = Number of blocks in current message
  $state \leftarrow H(state)$
  for $i \leftarrow 0$ to $counter$ **do**
    $N_i \leftarrow Trunc_{64}(state) \parallel i$ // N_i = Nonce used for block i
  end for
end while

Tronquer la sortie de la fonction de hachage entre $64 < X < 128 - MaxBlockLength$ est une autre solution, l'IV pourrait être extrait depuis le nonce lors de la réception d'un message. Toutefois, cet algorithme est moins sécurisé que le précédent puisqu'il possède un risque accru de collisions (taille du haché inférieure).

5.2.3 Générateur Congruentiel Linéaire

Un Générateur Congruentiel Linéaire (LCG) n'est généralement pas un générateur d'aléatoire utilisé en cryptographie (CSPRNG) pour la principale raison qu'il échoue le Next Bit test. En d'autres termes, cela signifie qu'un attaquant possédant les i premiers bits générés sera capable de prédire le bit $i + 1$ avec une probabilité supérieure à l'aléatoire (ici $1/2$).

Algorithm 4 LCG

$IV \leftarrow seed$
while true **do**
  $counter \leftarrow B$ // B = Number of blocks in current message
  for $i \leftarrow 0$ to $counter$ **do**
    $IV \leftarrow (a * IV + b)\ mod(m)$
    $Ni \leftarrow IV \parallel i$ // N_i = Nonce used for block i
  end for
end while

Cependant, une telle propriété n'est pas nécessaire dans le cadre de cette utilisation. Bien que le LCG ne fasse pas partie de la famille très restreinte des CSPRNG, en considérant une période suffisamment grande, il est tout à fait viable dans notre cas d'usage.

Note : Pour :

— m le module fixé à 2^{45}
— a le facteur fixé à 25214903917
— c l'incrément fixé à 11

seulement 0.024% – $2^{33}/2^{45} = 2^{-12}$ – de toutes les valeurs possibles générées par l'algorithme seront utilisées lors de la durée de vie de la clé AES.

5.3 Solution adoptée

Suite à la présentation effectuée aux équipes du projet, le générateur congruentiel linéaire a été maintenu en tant que générateur d'IV, la documentation du projet mise à jour et les alternatives ajoutées sur une partie annexe.

6 Conclusion

Ce stage a été une expérience riche et motivante menée en collaboration avec Tom Berland et toute l'équipe PAM et IAM de Capgemini Toulouse. Les sujets ont été proposés et non imposés et l'on été en fonction des différents parcours de chaque stagiaire, permettant ainsi la satisfaction de chacun sur le travail à réaliser durant la période définie. Ce projet a été l'occasion d'approfondir, de permettre une approche globale de la solution CyberArk et de la PAM, ainsi que de se mobiliser sur un travail conséquent qui a occupé la majeure partie de la durée de stage. Le sujet choisi à permis un grand nombre de recherches et apprentissage, source d'une grande satisfaction.

7 Glossaire

AD Active Directory, service développé par Microsoft sous les domaines Windows permettant de mettre en lien les utilisateurs (authentification et autorisation) avec les ressources dont ils ont besoin.

AES Advanced Encryption Standard, standard de chiffrement symétrique par blocs de 128 bits pour des clés de 128, 192 ou 256 bits.

Ansible Play Ensemble d'Ansible Tasks à jouer sur une ou plusieurs machines hôtes.

Ansible Playbook Fichier le plus haut hiérarchiquement composé d'un ou plusieurs Ansible Play(s).

Ansible Role Manière structurée d'organiser les Ansible Tasks, regroupant sous un même répertoire des fichiers de tâches visant un objectif similaire.

Ansible Task Action précise à jouer sur un hôte, les tâches pouvant être définies dans un Ansible Play ou dans un fichier de Tâches à inclure dans un Ansible Play via un rôle ou non.

CPM Central Policy Manager, composant permettant la gestion des secrets, la découverte de comptes à privilèges et la gestion de contrôle d'accès en fonction des politiques de sécurité.

CSPRNG Cryptographically Secure Psuedo Random Number Generator, fonction de génération psuedo-aléatoire ayant les propriétés suivantes : la sortie est imprédictible (satisfaction du Next Bit test), la fonction est résistante à une compromission partielle de son état initial.

FQDN Fully Qualified Domain Name est le nom de domaine complet d'une machine (ou hôte) sur internet. Il est composé du nom d'hôte (hostname) et du nom de domaine.

GPO Group Policy Objects, politiques appliquées à un groupe ou unité organisationnelle dans un Active Directory ou en local sur une machine (par exemple : droits utilisateurs, permissions d'accès, etc).

IaC Infrastructure as Code, capacité à mettre en place et maintenir une infrastructure informatique par du code et et non par un processus manuel.

IAM Identity Access Management, cadre composé de politiques et outils techniques assurant l'accès aux différentes ressources uniquement aux utilisateurs autorisés.

LDAP Lightweight Directory Access Protocol est un protocole standardisé permettant l'accès et la gestion d'un annuaire au moyen du protocole IP.

MFA Multi-Factor Authentication, authentification à multiples facteurs regroupant en général un savoir, une possession et/ou un attribut personnel permettant une identification plus sûre.

Microsoft IIS Internet Information Services, aussi appelé 2IS est le serveur web développé par Microsoft intégré au système d'exploitation Windows.

NTP Network Time Protocol, protocole réseau permettant la synchronisation des horloges locales sur une référence d'heure, donnée par le serveur.

PAM Privileged Access Management, brique de l'Identity governance and Administration (IGA) se concentrant sur les accès des comptes à privilèges.

PAS Privileged Access Security, solution permettant l'archivage, le transfert et le partage des mots de passe administrateurs par les utilisateurs autorisés.

pksc12 Format de fichier permettant de stocker plusieurs objets cryptographiques, communément utilisé pour les certificats X.509 et la clé privée associée.

PSM/P Privileged Session Manager / SSH Proxy, plateforme intermédiaire entre un serveur cible et un utilisateur. La connexion est établie après avoir requêté le Vault en fonction des droits de l'utilisateur en faisant la demande.

PTA Privileged Threat Analysis, outil d'analyse et de surveillance des comptes privilégiés enregistrés sur la plateforme CyberArk Privileged Access Manager - Self Hosted.

PVWA Password Vault Web Access, serveur Web permettant l'accès et la configuration du ou des PSM/P ainsi que les identifiants auquel chaque utilisateur peut prétendre.

Safes Sous compartiment du Vault contenant un ensemble de comptes, chaque utilisateur de la solution CyberArk possède l'accès à un ou plusieurs Safe(s) en fonction de ses droits.

Serveur cible Serveurs à administrer à l'aide d'un compte à privilèges.

SIEM Security Information and Event Management, outil de détection, d'analyse et de réponse à toute menace sur la sécurité d'un réseau ou d'une machine.

Vault Machine contenant tous les secrets des comptes instanciés sur les différents serveurs cible (mots de passe, clés ssh) ainsi que les politiques de sécurité et les sessions enregistrées par le PSM.

YAML Acronyme récursif de YAML Ain't Markup Language est un format de représentation de données visant à être le plus compréhensible et lisible possible pour un humain.

8 Annexes

8.1 Sources

- Documentation CyberArk : PAM self-hosted
- Documentation Ansible : How to create a small CA
- Github : Network Time Protocol Ansible role
- Github : Active Directory Ansible playbook
- CyberArk : PAS Orchestrator
- NIST 800 107 R1
- NIST 800 38 A
- Wikipedia : Next Bit test
- Wikipedia : Paradoxe des anniversaires
- Wikipedia : Principe des tiroirs
- Wikipedia : Registre à décalage à rétroaction linéaire

8.2 Documentation

La documentation fournie ci-dessous décrit le mode opératoire de déploiement (automatisé complet ou pas-à-pas). Celle-ci a été partagée avec le projet à toute l'équipe PAM et IAM afin de permettre la reprise du travail suite à la fin de ce stage.

Documentation

Vincent Joulain / Tom Berland

Août 2024

Table des matières

1 Arborescence du projet **2**

2 Prérequis **3**

3 Installation d'Ansible et récupération du projet **3**

4 Déploiement Automatisé complet **3**
4.1 Les variables d'environnement . . . 3
4.2 Le playbook . . . 4

5 Déploiement Automatisé pas-à-pas **5**
5.1 Mise en place de l'environnement . . . 5
5.1.1 déploiement des clés SSH . . . 5
5.1.2 déploiement des certificats . . . 5
5.1.3 Prérequis généraux . . . 5
5.2 Automatisation de l'Active Directory . . . 6
5.3 Déploiement de CyberArk . . . 6
5.3.1 Le Vault . . . 6
5.3.2 Le PrivateArk Client . . . 7
5.3.3 Le Password Vault Web Access . . . 7
5.3.4 Le Central Policy Manager . . . 7
5.3.5 Le Privileged Session Manager et Proxy . . . 8
5.3.6 L'HTML5 Gateway . . . 8
5.4 Onboarding . . . 8

6 Configuration des rôles **9**
6.1 Vault . . . 9
6.2 Password Vault Web Access . . . 9
6.3 Privileged Session Manager . . . 9
6.4 Certificates . . . 9
6.5 Active Directory . . . 9

7 Onboarding **10**
7.1 LDAP integration . . . 10
7.2 Local Accounts . . . 10
7.3 Autorisation du Master user . . . 10

8 Contraintes liées à l'environnement AzureLab **11**
8.1 Redémarrage des machines virtuelles après installation . . . 11
8.2 Durcissement du vault . . . 11

9 Améliorations de la solution : étapes restantes **11**
9.0.1 Génération du modèle AzureLAB . . . 11
9.0.2 Le Privileged Threat Analytics . . . 11
9.0.3 Onboarding . . . 12
9.0.4 Le Serveur Mail . . . 12

1 Arborescence du projet

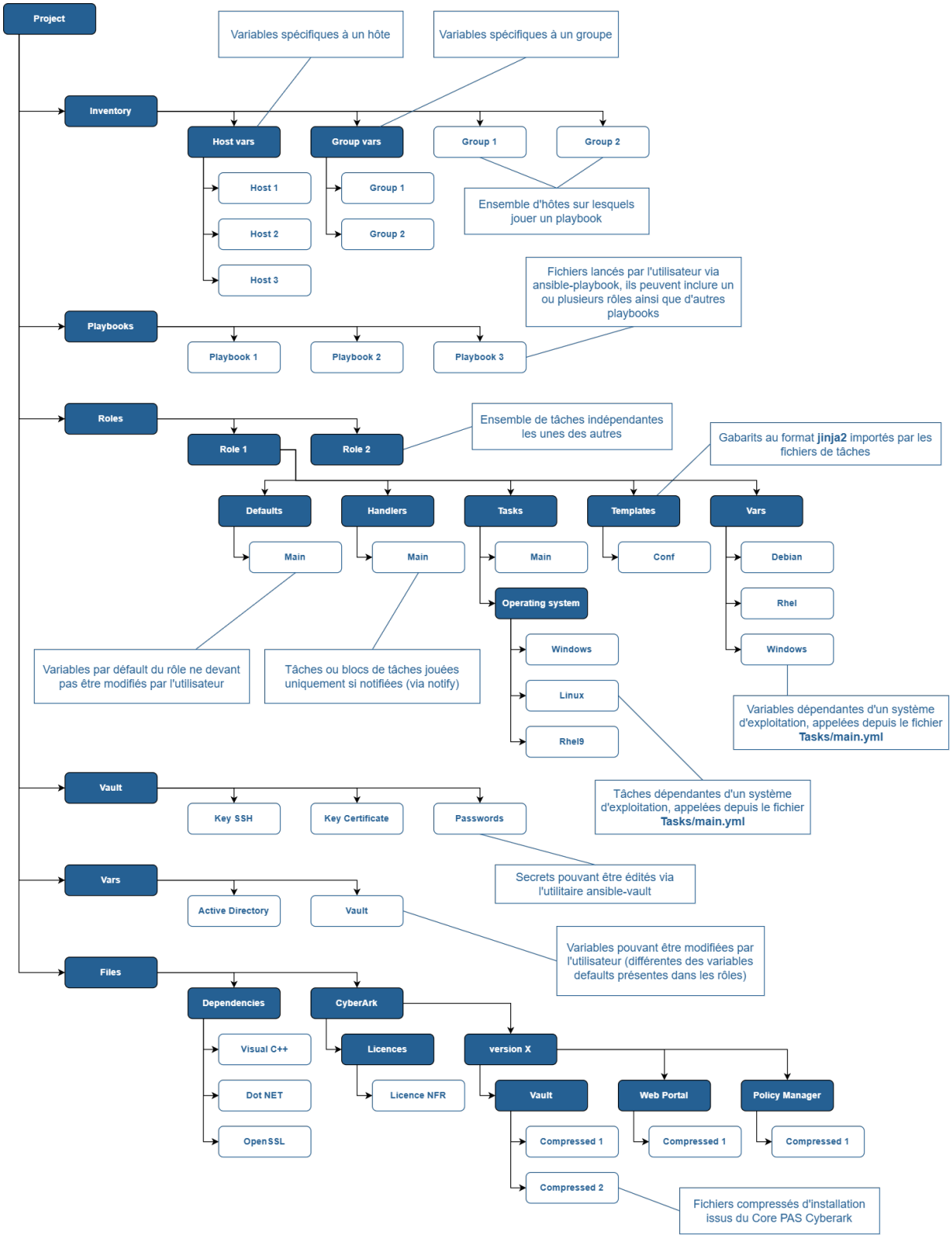


FIGURE 1 – Arborescence du projet

2 Prérequis

L'automatisation de l'installation est effectuée par l'outil Ansible, celui-ci nécessite sur chacun des hôtes :

— un compte super-utilisateur ayant le même identifiant / mot de passe sur toutes les machines
— un protocole de contrôle à distance (Open-ssh ou WinRM)

Les comptes super-utilisateurs étant déjà présents, il n'est nécessaire que d'installer et / ou configurer le protocole de contrôle à distance. Dans un souci de conformité, Open-ssh a été choisi.

Déjà préinstallé sur les machines Debian et Alma Linux, il a été installé sur les modèles AzureLAB Windows (mis à part le Vault primaire). Si nécessaire voir : Install Open-ssh on Windows

3 Installation d'Ansible et récupération du projet

Télécharger le github du projet (si disponible) ou récupérer celui-ci depuis le Sharepoint présent dans **Communauté PAM France**. Créer ensuite l'environnement virtuel python et installer Ansible sur l'IaC Worstation :

```
# ajout des droits d'éxecution
cd Deploiement-CyberArk-via-Ansible && chmod +x conf/scripts/ansible_install.sh

# lancement du script d'installation
./conf/scripts/ansible_install.sh
```

4 Déploiement Automatisé complet

4.1 Les variables d'environnement

La première étape avant toute utilisation du playbook est de définir les variables nécessaires au déploiement de CyberArk.

Éditer le fichier **inventory/group_vars/all.yml** après avoir récupéré les adresses IP de chacune des machines :

```
# IP Adresses
ansible_workstation: "<IP Workstation>"          target_win: "<IP Target Linux>"
domain_controller: "<IP Domain Controller>"      target_lin: "<IP Target Windows>"
cpm: "<IP Central Policy Manager>"               pta: "<IP Privileged Threat Analytics>"
vault: "<IP Vault>"                              psmp: "<IP PSM Proxy>"
vault_DR: "<IP Vault Disaster Recovery>"         pvwa: "<IP Password Vault Web Access>"
psm: "<IP Privileged Session Manager>"           mfa: "<IP MFA server>"
```

Modifier la variable **ansible_user** si nécessaire (nom des comptes locaux ayant les droits administrateurs utilisés par Ansible afin d'effectuer les étapes d'installation sur chacune des machines) :

```
ansible_user: defender
```

Ouvrir le fichier de mots de passe **vault/passwords.yml** à l'aide de la commande suivante (le mot de passe est *"changeme"*) :

```
ansible-vault edit vault/passwords.yml
```

Entrer les secrets permettant d'accéder aux machines ainsi que les mots de passe initiaux des comptes CyberArk :

```
ansible-vault edit vault/passwords.yml
```

L'installation par défaut ne requiert pas plus de modifications. Afin d'apporter des changements à celle-ci, se référer à la partie concernée dans Déploiement Automatisé pas-à-pas

4.2 Le playbook

Le playbook **main.yml** permet l'installation complète de l'environnement CyberArk incluant les 12 machines suivantes :

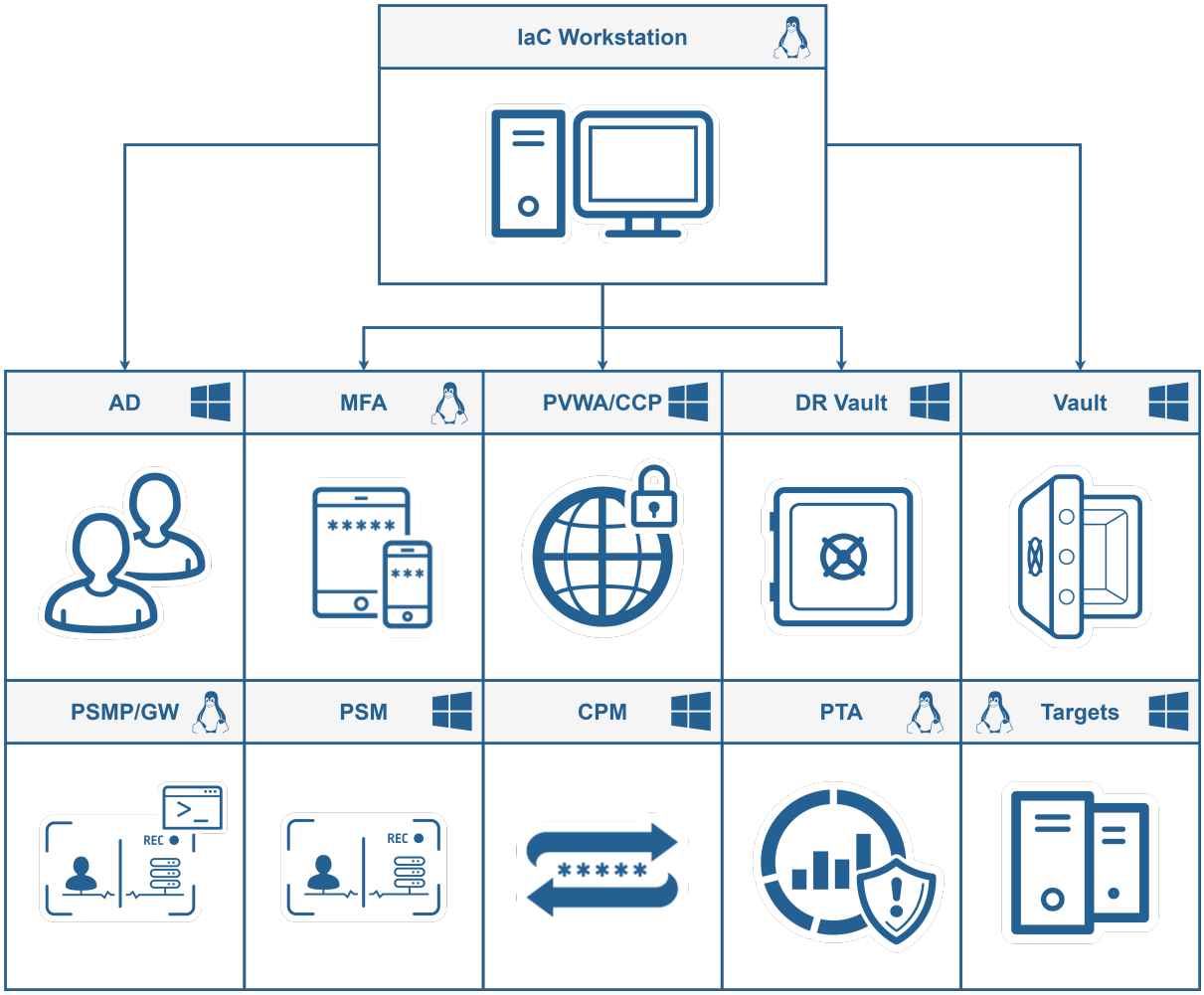


FIGURE 2 – Architecture AzureLAB

Lancer le playbook de déploiement :

```
# Déploiement de l'architecture complète
ansible-playbook playbooks/main.yml --ask-vault-pass
```

5 Déploiement Automatisé pas-à-pas

5.1 Mise en place de l'environnement

5.1.1 déploiement des clés SSH

L'utilisation de SSH par mot de passe n'est pas une bonne pratique de sécurité, il est recommandé d'utiliser des paires de clés publiques / privées afin de limiter les risques d'intrusion. Le paramètre **–ask-vault-pass** permet de récupérer le mot de passe des comptes super-utilisateur (dans le fichier **vault/passwords.yml**) afin d'effectuer la première connexion, les suivantes utilisant les clés mises en place.

Il est possible pour la suite de simplifier les commandes d'exécution des playbooks à l'aide de la variable **ask_vault_pass** présente dans **ansible.cfg**. Celle-ci permet de déchiffrer le fichier de mots de passe afin de fournir l'accès aux variables lors de l'exécution de chaque playbook. Ce changement n'a pas été effectué afin d'éviter que les playbook n'en ayant pas le besoin n'accèdent à ces variables.

Lancer le playbook **secure_shell.yml** :

```
# Génération de la clé SSH et déploiement de celle-ci sur les machines cibles
ansible-playbook playbooks/secure_shell.yml --ask-vault-pass
```

Le playbook ci-dessus permet de générer une clé SSH (ed25519) et d'ajouter la clé publique liée dans le fichier **∼/.ssh/authorized_keys** sous Linux et **C :\ProgramData\ssh\administrators_authorized_keys** sous Windows.

5.1.2 déploiement des certificats

Dans l'optique d'une utilisation future du protocole WinRM et d'éviter les avertissements de sécurité du navigateur web, il est intéressant de mettre en place une autorité de Certification qui distribuera des certificats aux différents serveurs.

Lancer le playbook **certificates.yml** :

```
# Déploiement du certificat racine et certificats serveurs
ansible-playbook playbooks/certificates.yml
```

Le playbook est divisé en quatre rôles différents :

- Génération d'une clé permettant de créer le certificat racine auto-signé.
- Génération des différentes clés et certificats serveur et signature de ceux-ci avec la clé de l'autorité de certification.
- Partage du certificat racine en tant qu'autorité de certification de confiance dans les magasins des hôtes Windows et Linux.
- Lien du certificat serveur au site internet (PVWA uniquement).

Attention : Le playbook de gestion des certificats doit être lancé après celui du PVWA (intégration du certificat sur le portail web) mais avant celui du HTML Gateway (utilisation du certificat afin de générer l'instance Podman).

5.1.3 Prérequis généraux

Avant de pouvoir effectuer les installations correctes et joindre au domaine les hôtes, il est nécessaire de les renommer ainsi que de mettre en place le serveur DNS ; sur les machines Linux, il est essentiel de vérifier la présence des paquets requis et les installer le cas échéant.

Lancer le playbook **prerequisites.yml** :

```
# Prérequis généraux s'éxecutant sur toutes les hôtes
ansible-playbook playbooks/prerequisites.yml
```

5.2 Automatisation de l'Active Directory

L'active Directory est installé selon le modèle suivant :

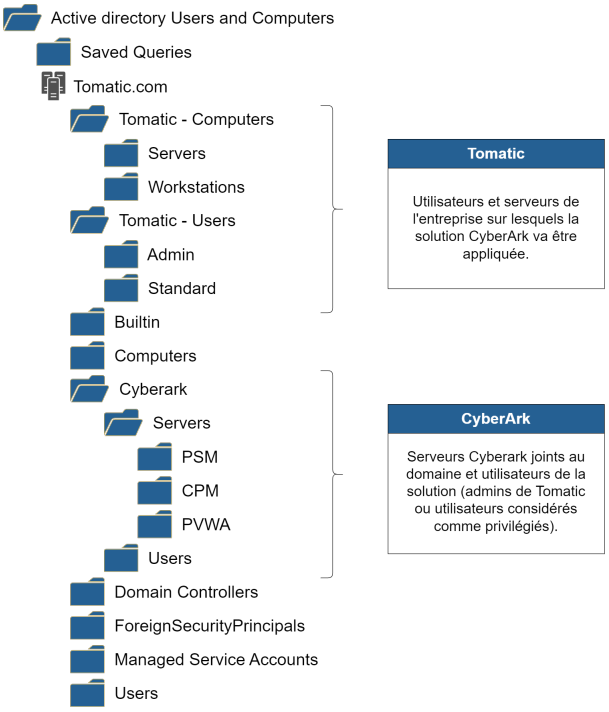


FIGURE 3 – Active Directory

Le role **active_directory** est divisé en plusieurs parties, l'initialisation du Primary Domain Controller (PDC), l'installation du schéma (unités d'organisation, groupes et utilisateur) ainsi que la jointure au domaine des machines requises (PVWA, CPM, PSM).

```
# Déploiement de l'AD, ajout des groupes/utilisateurs et jointure au domaine des machines Windows
ansible-playbook playbooks/active_directory.yml --ask-vault-pass
```

5.3 Déploiement de CyberArk

5.3.1 Le Vault

Le Vault est l'élément central de l'infrastructure CyberArk, les ressources permettant de le déployer sont présentes dans le dossier **files/** à télécharger et a ajouter à la racine de la structure du projet. Ces ressources ont été récupérées depuis le market place de CyberArk (le PrivateArk Client n'est plus présent dans le CorePAS de CyberArk depuis la version 14)

Le rôle du Vault peut-être importé de deux manières différentes en fonction des variables booléennes **vault_install** et **vault_disaster_recovery** permettant respectivement de lancer l'installation classique et / ou l'installation Disaster Recovery (à condition que l'hôte soit désigné comme **disaster_recovery_host** dans les variables d'hôtes). Si lancé sans aucune variable, le rôle n'exécutera aucune tâche.

Attention : L'installation Vault Disaster Recovery nécessite la mise en place de l'API CyberArk et par conséquent du **PVWA**. Il ne faut lancer le rôle du **vault_DR** qu'une fois le **PVWA** installé.

Modifier le playbook **vault.yml** afin de lancer l'installation classique sur le **vault** et **vault_DR** :

```
- name: Deploy vault
  hosts: vault, vault_DR
  vars_files:
    - ../vault/passwords.yml
  vars:
    vault_install: true
  roles:
    - role: ../roles/vault/
```

Lancer le playbook **vault.yml** :

```
# Déploiement du vault
ansible-playbook playbooks/vault.yml --ask-vault-pass
```

Le **PVWA** installé, le **vault_DR** peut à son tour être instancié de la manière suivante :

```
- name: Deploy vault
  hosts: vault, vault_DR
  vars_files:
    - ../vault/passwords.yml
  vars:
    vault_disaster_recovery: true
  roles:
    - role: ../roles/vault/
```

```
ansible-playbook playbooks/vault.yml --ask-vault-pass
```

5.3.2 Le PrivateArk Client

Le client PrivateArk est l'outil permettant d'accéder et d'administrer directement le Vault , il doit être installé d'abord sur le Vault puis sur les composants CyberArk choisis afin d'en permettre la gestion à distance.

Lancer le playbook **privateark_client.yml** :

```
# installation du client sur les vaults
ansible-playbook playbooks/privateark_client.yml
```

5.3.3 Le Password Vault Web Access

Le Password Vault Web Access, aussi appelé portail web ou PVWA, est le site internet interne de la solution CyberArk auquel les utilisateurs doivent se connecter afin d'avoir accès à des comptes privilégiés. Le rôle du PVWA contient les fichiers permettant l'installation du portail web ainsi que du Central Credential Provider (CCP) souvent installé sur la même machine.

Lancer le playbook **password_vault_web_access.yml** :

```
ansible-playbook playbooks/password_vault_web_access.yml  --ask-vault-pass
```

Attention : Par défaut les scripts d'installation CyberArk génèrent un certificat auto-signé. Il faut penser à jouer le playbook **certificates.yml** suite à l'installation du PVWA afin d'assigner un nouveau certificat serveur et permettre aux autres composants de l'authentifier.

5.3.4 Le Central Policy Manager

Le CPM est un composant essentiel pour la gestion des secrets. Il automatise les tâches de gestion des mots de passe telles que la rotation, la vérification et la réconciliation sur les comptes cibles. Il est uniquement accessible aux administrateurs CyberArk. De ce fait, c'est sur ce composant qu'il est possible d'installer le client PrivateArk.

Lancer le playbook **central_policy_manager.yml** :

```
ansible-playbook playbooks/central_policy_manager.yml  --ask-vault-pass
```

5.3.5 Le Privileged Session Manager et Proxy

Le PSM/P est un outil crucial pour la sécurité des sessions administrateurs. Il permet de sécuriser, contrôler et surveiller les accès privilégiés aux appareils réseau. Les sessions établies sur les systèmes cibles sont entièrement isolées et les identifiants des comptes privilégiés ne sont jamais exposées aux utilisateurs finaux. De plus, des audits détaillés ainsi que des enregistrements vidéo de toutes les sessions privilégiées sont sauvegardés.

Lancer le playbook **privileged_session_manager.yml** :

```
ansible-playbook playbooks/privileged\_session\_manager.yml --ask-vault-pass
```

Lancer le playbook **privileged_session_manager_over_ssh.yml** :

```
ansible-playbook playbooks/privileged_session_manager_over_ssh.yml --ask-vault-pass
```

5.3.6 L'HTML5 Gateway

L'HTML5 Gateway est un composant Cyberark permettant à un utilisateur d'ouvrir une connexion vers le PSM au travers d'un websocket sur le port 443 (https). Il est ainsi possible de se connecter à une machine cible sans passer par du RDP mais uniquement à l'aide d'un navigateur web.

Lancer le playbook **html_gateway.yml** :

```
ansible-playbook playbooks/html_gateway.yml  --ask-vault-pass
```

5.4 Onboarding

l'embarquement des comptes est divisé en deux sous-rôles : intégration LDAP et ajout des comptes locaux des machines serveur.

La liaison des comptes Active Directory dans le Vault se fait à l'aide de l'intégration LDAP. Dans ce cas, les utilisateurs des groupes **Vault Admins**, **Auditors**, **Users** et **Safe Managers** de l'AD sont liés aux rôles CyberArk du même nom. Des droits d'accès sont assignés en fonction des autorisations spécifiées. Afin de permettre l'intégration, un utilisateur Domain Admin doit être utilisé.

L'utilisation des comptes locaux requiert d'activer l'option **Require privileged session monitoring and isolation** sur le PVWA ¿ Policies ¿ Master Policy ¿ Session Management afin de permettre la connexion au travers du PSM.

Attention : Par manque de temps, l'intégration LDAPS n'a pas été mise en place et par conséquent les échanges ne sont pas sécurisés.

Lancer le playbook **onboarding.yml** :

```
ansible-playbook playbooks/onboarding.yml --ask-vault-pass
```

6 Configuration des rôles

Ajouter les variables ci-dessous dans le fichier **inventory/group_vars/all.yml** afin d'écraser leur valeur par défaut si nécessaire.

6.1 Vault

IP source des connexions RDP autorisées :

```
client_ip_range: "0.0.0.0-255.255.255.255"
```

6.2 Password Vault Web Access

Méthodes d'authentification acceptées :

```
PVWA_auth_type: "CyberArk;LDAP;"
```

Déploiement de la version classique et/ou mobile :

```
PVWA_enable_full_version: true
PVWA_enable_mobile_version: true
```

Méthode d'authentification par défaut :

```
PVWA_default_auth_method: "CyberArk"
PVWA_default_mobile_auth_method: "CyberArk"
```

6.3 Privileged Session Manager

Intégration ou non au domaine :

```
psm_out_of_domain: false
```

6.4 Certificates

Common name du certificat racine :

```
common_name: Tomatic S.A.R.I
```

6.5 Active Directory

La configuration de l'Active Directory, relativement complexe, est disponible dans le fichier **vars/active_directory.yml**. Celui-ci contient dans l'ordre suivant : les unités d'organisation, les groupes, les machines puis utilisateurs.

A noter : Il est impératif de créer un utilisateur possédant au minimum les droits de lecture sur l'Active Directory afin de permettre le mapping CyberArk. Les groupes **Vault Admin**, **Auditors**, **Users** et **Safes Managers** ont été créés et seront intégrés respectivement aux groupes CyberArk du même nom lors de l'étape d'onboarding. Les profils utilisés lors des formations CyberArk (Mike, Cindy, Carlos, Paul, Tom, John et Dexter) ont été répartis dans ces groupes.

Afin de joindre une machine CyberArk au domaine, les chemins doivent être spécifiés en tant que variable d'hôte **domain_path** :

```
domain_path: "OU=CPM,OU=Servers,OU=CyberArk,DC={{ ad_domain_name }},DC={{ ad_extension }}"
```

7 Onboarding

7.1 LDAP integration

Variables d'intégration des groupes LDAP :

```
#list of mapping authorizations
#AddSafes
#AuditUsers
#AddUpdateUsers
#ResetUsersPasswords
#ActivateUsers
#AddNetworkAreas
#ManageServerFileCategories
#BackupAllSafes
#RestoreAllSafes

mapping_authorization:
  - { mapping_name: '...',
      LDAP_branch: '...',
      domain_groups: '[ ... ]',
      groups: '...',
      mapping_authorizations: '[ ... ]',
      ...
    }
```

7.2 Local Accounts

Safes Cyberark à créer :

```
safes:
  - WinAdmins
  - LinAdmins
```

Comptes locaux à ajouter :

```
local_accounts:
  - { name: ...,
      address: ...,
      userName: ...,
      platformId: ...,
      safeName: ...,
      secret: ...,
      secretType: ...
    }
```

7.3 Autorisation du Master user

Le paramètre et la valeur de la clé Master se trouvent dans le fichier dbparm.ini, placer le chemin d'accès à la clé Master (dans les fichiers d'installation du Vault) :

```
RecoveryPrvKey="Drive:\RecPrv.key"
```

8 Contraintes liées à l'environnement AzureLab

Certaines contraintes spécifiques à l'environnement AzureLAB ont du être prises en compte tout au long du projet. Les prochains paragraphes présentent chacune des contre-mesures apportées afin de faciliter l'export de la solution d'automatisation sur un autre environnement.

8.1 Redémarrage des machines virtuelles après installation

Lors du déploiement des composants CyberArk au moyen de l'assistant d'installation, il est parfois possible de cocher une case permettant le redémarrage automatique de la machine. En revanche, il est important lors de la génération des fichiers de réponse de ne pas cocher cette case. Le redémarrage machine doit passer par Ansible au risque de perdre la connexion établie avec la station principale. Par conséquent, les reboot sont effectués de la manière suivante :

```
- name : Reboot
  ansible.windows.win_reboot:
  when: log_result.stdout | from_json | community.general.json_query('restartRequired')
```

8.2 Durcissement du vault

Le durcissement du vault a posé de nombreux problèmes dans le contexte AzureLAB notamment à cause de certains fonctionnements internes. Dans un premier temps, il est nécessaire de réinitialiser la variable de registre **fPromptForPassword** (always prompt for password) sans quoi la connexion à l'aide de mots de passe enregistrés n'est plus autorisée entraînant la non possibilité pour les serveurs d'AzureLAB d'initier la première connexion.

```
- name: Vault Harden - Group policies, disable always prompt for password (AzureLAB)
  ansible.windows.win_regedit:
    path: HKLM:\SOFTWARE\Policies\Microsoft\Windows NT\Terminal Services
    name: fPromptForPassword
    data: 0
    type: dword
```

Enfin, afin d'éviter de bloquer les flux **heartbeat** réguliers envoyés par les machines virtuelles au serveur AzureLAB, l'étape de durcissement du pare-feu a été désactivée :

```
- name: Vault harden - Disable firewall hardening (AzureLAB)
  community.windows.win_lineinfile:
    path: "{{ private_ark_folder }}\\Server\\Hardening\\Conf\\Hardening.ini"
    regexp: "^HardenWindowsFireWall="
    line: HardenWindowsFireWall=No
```

9 Améliorations de la solution : étapes restantes

9.0.1 Génération du modèle AzureLAB

Le modèle AzureLAB n'a pas encore été mis en place dans sa globalité.

Afin de permettre le lancement efficace de la solution, il est nécessaire de :

- changer le système d'exploitation de la machine **Defender-PSMP-PSMGW** pour un almalinux version 9 ou supérieur
- installer et activer le lancement au démarrage du service openssh-serveur sur la machine **Defender-Vault-01**

9.0.2 Le Privileged Threat Analytics

Le PTA peut être installé à l'aide du rôle présent dans la solution et tiré de celui proposé par CyberArk. L'installation s'effectue correctement dans sa globalité, cependant, l'étape de configuration échoue avec l'erreur suivante **status = RUNNING instead SUCCESS**.

9.0.3 Onboarding

Les sous-rôles **onboarding_rules** et **users_and_groups** sont des ébauches de playbook disponibles dans le rôle onboarding. Ceux-ci permettent d'avoir une idée des commandes API à utiliser mais aucun test n'a encore été réalisé.

9.0.4 Le Serveur Mail

Le serveur mail fonctionne à l'aide de l'Event Notification Engine, un service du Vault. Tout comme l'Active Directory, il est recommandé de créer un utilisateur dans le groupe **Vault Admins** afin d'obtenir les permissions requises pour établir ce service.

La mise en place d'un serveur mail s'effectue en 3 étapes :

— Installation du serveur SMTP sur le contrôleur de domaine, tâche effectuée depuis le **Server Manager** ce dernier faisant partie des fonctionnalités Windows.

— Choix, installation et configuration d'un client mail. MailEnable est suffisant et simple d'installation, cependant n'importe quel autre client peut être utilisé.

— Intégration du serveur mail à CyberArk. Étape problématique de la mise en place du service mail puisqu'aucune méthode d'automatisation n'a été trouvée sur la documentation officielle. Deux pistes de recherche demeurent : la première serait l'utilisation du CyberArk command line interface (PACLI), La seconde de vérifier quels fichiers de configuration sont modifiés lorsque l'intégration est faite manuellement (à l'aide d'une fonction de hachage).

L'intégration manuelle s'effectue depuis le PVWA en lançant le **setup.exe** situé dans la partie configuration option à gauche dans le menu. Il est nécessaire de renseigner :

— l'IP de la machine où est installé le serveur SMTP
— le nom de l'expéditeur
— l'adresse mail de l'utilisateur envoyant les mails (membre de Vault Admin).